

UNIVERSITATEA TEHNICĂ „Gheorghe Asachi” din IAȘI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DOMENIUL: Calculatoare și tehnologia informației
SPECIALIZAREA: Calculatoare

Sisteme multimedia

material de curs

Cuprins

Introducere.....	1
Terminologia cuvântului multimedia.....	1
Definiții.....	1
Aplicații ale „multimediei”.....	2
Capitolul 1. Elemente ale comunicației multimedia.....	4
1.1. Comunicația audio.....	5
1.1.1. Natura sunetului.....	5
1.1.2. Proprietăți fizice ale sunetului.....	6
1.1.2.1. Frecvența.....	7
1.1.2.2. Lungimea de undă.....	7
1.1.2.3. Efectul Doppler.....	7
1.1.2.4. Banda de frecvențe.....	8
1.1.2.5. Armonici.....	8
1.1.2.6. Viteza sunetului.....	9
1.1.2.7. Intensitatea sunetului.....	9
1.1.3. Parametri psiho-acustici.....	9
1.1.3.1. Înălțimea sunetului.....	10
1.1.3.2. Nivelul sunetului.....	11
1.1.3.3. Localizarea sunetului.....	12
Capitolul 2. CODEC-uri video.....	14
2.1. Scurtă istorie a CODEC-urilor video.....	14
2.2. Principiile compresiei video.....	16
2.2.1. Reducerea redundanțelor spațiale.....	17
2.2.1.1. Codarea predictivă.....	17
2.2.1.2. Codarea prin transformare.....	17
2.2.1.3. Erori de aproximare.....	19
2.2.1.4. Transformata cosinus discretă rapidă.....	20
2.2.1.5. Transformatele subband și wavelet.....	24
2.2.1.6. Cuantizarea coeficienților.....	25
2.2.2. Reducerea redundanțelor temporale.....	26
2.2.2.1. Estimarea mișcării.....	27
2.2.3. Codare pe lungime variabilă de biți.....	28
2.2.3.1. Codarea Huffman.....	28
2.2.3.2. Codarea Huffman canonică.....	30
2.2.3.3. Arbori Huffman cu adâncime limitată.....	33
2.2.3.4. Codificarea aritmetică.....	35
2.2.3.5. Codificarea aritmetică adaptivă.....	38
Bibliografie.....	39

Introducere

Multimedia este în contextul contemporan un concept general folosit pentru a indica orice combinație de metode distincte de comunicare, inclusiv radio, film, televiziune, teatru, precum și alte tipuri de spectacol *live*. Termenul este mai ales asociat cu sistemele de calcul, deoarece acestea sunt de obicei utilizate pentru a angaja simultan diferite metode de comunicare (de exemplu, video, audio, imagini, text, etc.).

Termenul de multimedia este relativ tânăr, acesta a apărut pentru prima dată în anii 1960, și a avut de-a lungul timpului mai multe semnificații. Conceptul de multimedia precede cu mulți ani apariția termenului de multimedia. De exemplu, punerea în scenă a operei lui Jules Verne „Ocolul Pământului în 80 de zile” în anul 1874 a fost o *féerie* (un gen teatral francez, care utiliza efecte mecanice și scenice grandioase), care a jucat pe scena teatrului din Châtelet pentru mai bine de cincizeci de ani. Acesta a fost numit multimedia *avant la lettre* (din franceză: înainte de timpul său, avangardist) deoarece acesta a utilizat simultan un număr considerabil de muzicieni, scenografi și actori.

Terminologia cuvântului multimedia

Termenul *multimedia* a fost inventat de cântărețul și artistul Bob Goldstein pentru a populariza în revista *Variety* natura tehnică a spectacolului său numit *LightWorks* care a avut loc în 1965 în Long Island și New York City [1]. În următorii patruzeci de ani, cuvântul a căpătat sensuri diferite, spre exemplu la sfârșitul anilor 1970, termenul se referea la prezentări compuse din proiecții video statice sincronizate cu o bandă audio. Abia în anii 1990, termenul *multimedia* a căpătat înțelesul său actual.

Pentru a înțelege semnificația termenului de *multimedia* putem pleca de la etimologia cuvântului. Termenul *multimedia* provine din latinescul *multum* și *medium*. Este un mediu care utilizează o „multitudine” de „medii” diferite. Termenul de *multimedia* poate fi definit ca o combinație de două sau mai multe medii de comunicație pentru a transmite o idee (cum ar fi: text, imagine, sunet, vorbire, video și aplicații pe computer). Astfel *multimedia* îmbunătățește procesul de comunicație și de înțelegere.

Definiții

În 1992 Blattners și Dannenberg defineau un sistem informatic multimedia ca fiind un sistem capabil de achiziție a datelor de intrare și de afișare a datelor de ieșire utilizând mai multe medii. De obicei, termenul se aplică sistemelor care suportă mai mult de un singur mediu fizic de ieșire, cum ar fi un monitor, sisteme video și audio [2]. Tot aceștia fac în continuare observația că *sistemele multimedia* se străduiesc să țină cont de avantajele simțurilor umane, cu scopul de a facilita comunicarea. Astfel prin „multimedia” comunicatorul alege să îmbine mai multe medii de comunicare care sunt cele mai potrivite pentru transmiterea mesajului dorit și care oferă cea mai o mai bună experiență de comunicare.

Tot în 1992 Tay Vaughan în prima ediție a cărții sale „Multimedia: Making It Work” definea multimedia ca fiind „orice combinație de text, artă, sunet, animație și video trimisă către utilizator prin intermediul calculatorului sau a altor mijloace electronice de manipulare a datelor digitale”. Acesta este o experiență senzorială îmbunătățită. Când sunt întreșute elementele senzoriale ale multimediei – imagini și animații scilpitoare, sunete captivante, clipuri video atrăgătoare și informații text – se poate obține un efect electrizant minții receptorului. Dacă acesta are și control interactiv asupra procesului, receptorul uman poate fi chiar fermecat [3].

Enciclopedia Britanică definește *multimedia interactivă* ca fiind orice sistem de calcul ce permite utilizatorului să controleze, să combine și să manipuleze diferite tipuri de media. În această definiție, accentul este pus pe interactivitate și pe controlul asupra computerului pentru

furnizarea de informații în diferite medii. Cititorul sau privitorul are control asupra mediilor de comunicație, astfel încât acestea să poată participa la dezvoltarea „produsului” multimedia prin interacțiunea cu acesta.

În DEX¹ termenul multimedia este definit ca „ansamblu de tehnici (texte, imagini fixe sau animate, sunete, grafică) care permit recepționarea, depozitarea, prezentarea și prelucrarea informațiilor simultan și interactiv, fiind folosit în scopuri educative sau ca divertisment”.

Pe baza descrierilor anterioare putem defini *multimedia digitală* ca fiind o combinație de text, fotografii, grafică, sunet, animație și video întrețesute prin mijloace de manipulare digitală a datelor. Atunci când utilizatorul final are și control asupra informației multimedia livrate – aceasta este numită *multimedia interactivă* sau „*rich media*”. Când elementele informației multimedia sunt oferite utilizatorului într-o structură prin care utilizatorul poate naviga, multimedia interactivă devine *hipermedia*.

O aplicație multimedia universală este privită în [4] drept una care manipulează tipuri de date ce pot fi combinate într-un document, afișate pe ecran sau tipărite, fără manipulări speciale pe care utilizatorul trebuie să le efectueze. Media care compune documentul poate fi obținută din surse sau servere distribuite într-o rețea și va trebui să fie sincronizată înainte de a fi livrată utilizatorului. În general, sincronizarea multimedia denotă o relație temporală, spațială sau chiar logică între obiecte, entități de date sau fluxuri de date media [5]. În contextul calculatoarelor multimedia și a comunicațiilor, totuși, în cele mai multe cazuri prin sincronizare este abordată doar relația temporală.

Instrumentul software, mesajele și conținutul prezentat pe un calculator personal, ecran de televiziune, tablete, telefoane mobile, etc. constituie împreună un *proiect multimedia*. În cazul în care proiectul este expediat sau vândut către utilizatori finali, ca un pachet ce poate fi descărcat de pe internet sau de pe un CD-ROM sau DVD, cu sau fără instrucțiuni, acesta este un *titlu multimedia*.

Un proiect multimedia nu trebuie să fie interactiv pentru a putea fi numit multimedia; este cazul proiecțiilor cinematografice în care utilizatorii nu pot asimila informația în ritmul lor propriu. Un astfel de proiect este *liniar* este derulat într-un singur sens, de la început către sfârșit. Atunci când utilizatorii au posibilitatea de a controla proiectul și de a naviga nestingheriți prin conținutul acestuia, multimedia devine *neliniară* și interactivă.

Elementele multimedia îmbinate într-un proiect prin intermediul unor instrumente software special concepute pentru a gestiona elementele individuale ale conținutului multimedia și să ofere interacțiune cu utilizatorul. Cele mai multe dintre aceste instrumente software oferă și posibilitatea creării atât a informației multimedia propriu-zice cât și a instrumentului software prin intermediu căruia asta poate fi accesată de către utilizator. Suma metodelor prin care se redă conținutul proiectului și prin care se alege modul în care este prezentat utilizatorului pe ecran este denumită *interfața grafică cu utilizatorul* sau *GUI* (*graphical user interface*). Aceasta conține de cele mai multe ori ustensilele prin care utilizatorul își exercită controlul asupra conținutului multimedia.

Aplicații ale „multimediei”

Multimedia se pretează în orice context în care utilizatorul uman are contact cu informația digitală prin intermediul dispozitivelor electronice prin așa numita „interfața cu utilizatorul uman”. Multimedia îmbunătățește comunicația om-calculator prin câștigarea și păstrarea atenției și interesului utilizatorului. Atunci când este construită în mod corespunzător, multimedia poate fi simultan distractivă și utilă.

Sistemul de învățământ este poate cel care poate beneficia cel mai mult de aplicații multimedia. Învățământul ca sistem subfinanțat și lent în a adopta noi tehnologii este sistemul în care multimedia poate fi maximizată pentru a oferi beneficii evidente pe termen lung.

1 DEX este Dicționarul EXplicativ al limbii române, care conține termeni și definiții în limba română.

Multimedia va provoca schimbări radicale în procesul de predare în viitor, limitele metodelor tradiționale de predare fiind evidente. Se observă tendința de trecere de la modelul de învățare pasiv la cel activ, prin experimentare. Platformele de învățare online de tip *E-learning* sunt un subiect sensibil, aplicațiile software educaționale fiind văzute mai degrabă ca sisteme de învățământ complementare și nu un potențial substitut al modelului tradițional.

De la grădărit, gătit până la proiectarea, remodelarea și reparația locuințelor, multimedia, a intrat în casele noastre. Cele mai multe proiecte multimedia ne ajung în acasă prin intermediul aparatelor de televiziune, calculatoarelor, tabletelor, telefoanelor etc.. Aplicațiile multimedia la locul de muncă includ prezentări, programe de formare, marketing, publicitate, simulări, baze de date, cataloage, mesagerie și de comunicații prin rețeaua locală, internet sau cea de telefonie.

Industria de divertisment este domeniul care în ziua de azi nu mai poate fi conceput separat de cel de multimedia. Industria filmelor artistice, piese de teatru, concerte de muzică, jocuri pe calculator, etc. toate sunt exemple vii ale aplicațiilor multimedia. Sume colosale de bani se „învârt” în industria de divertisment, iar aceasta impulsionează și încurajează direct sau indirect dezvoltarea de noi tehnici multimedia și noi tehnologii pentru crearea, stocarea și redarea de conținut multimedia.

Există două modalități prin care putem înțelege multimedia. Prima este să vedem multimedia prin prisma definițiilor, istoriei sale, exemplelor și problemelor teoretice – acest studiul academic al domeniului multimedia se concretizează prin accesarea de manuale, articole și prelegeri. A doua modalitate este de a utiliza efectiv „multimedia” pentru a ne „transmite” gândurile. A utiliza multimedia e înseamnă a explora idei noi și a le comunica. Într-un domeniu cum ar fi multimedia, este tot atât de important găsirea de idei cât și identificarea de modalități noi de a le prezenta pe cele vechi. Multimedia este un mod de a gândi.

Capitolul 1. Elemente ale comunicației multimedia

Oamenii percep universul prin intermediul simțurilor. Aceste simțuri – vederea și auzul în special - sunt esențiale în interacțiunea cu mediul înconjurător. Organele de simț trimit semnale către creier, pe baza cărora acesta construiește o interpretare a acestei interacțiuni. Procesul de comunicare, de a trimite mesaje de la o persoană la alta, depinde de modalitatea în care sunt interpretate simțurile. În general, cu cât mai multe informații sunt percepute de către receptor, cu atât este mai probabil ca o comunicare eficientă să aibă loc. Astfel, cu cât mai multe informații sunt transmise, cu atât mai mare este impactul comunicării.

Mediul de comunicare – de exemplu, o scrisoare sau un apel telefonic – limitează utilizarea diferitelor elemente. Dezvoltarea dispozitivelor de comunicații vizează creșterea cantității de informație care poate fi transmisă. Dezvoltarea sistemelor de calcul se mișcă și în această direcție. Astfel, la început, calculatoarele erau capabile să ofere doar simplu text ca ieșire, acum generează sunete, afișează imagini 2D sau 3D și multe altele.

Multimedia digitală este definită ca orice combinație de text, imagini grafice (statice sau animate), sunet și imagini în mișcare (animații sau video) livrate de un sistem de calcul. Sistemele de calcul constituie o parte intrinsecă a comunicației multimedia. Toate aceste elemente – text, grafică, sunet și video – sunt fie generate de calculator, fie transmise prin intermediul acestuia.

Cea mai importantă diferență dintre mediile multimedia tradiționale (mass-media), cum ar fi radioul și televiziunea și multimedia digitală, este noțiunea de interactivitate. Sistemele de calcul permit utilizatorilor să interacționeze cu programele. Deoarece interactivitatea este un concept atât de puternic, mulți experți în domeniul multimedia consideră că interactivitatea este o parte integrantă a „multimediei”. Astfel, în prezent, prin cuvântul multimedia, se înțelege multimedia digitală interactivă.

Astfel comunicația multimedia constă din mai multe elemente:

- comunicația audio;
- comunicația video;
- comunicația text.

Primul element al comunicației multimedia este audio. Cuvântul audio are o semnificație puțin diferită față de sunet. Sunetul constă în sunetele pe care oamenii le pot auzi. Acest lucru poate părea contradictoriu, dar este bine cunoscut faptul că oamenii nu aud toate sunetele posibile. De exemplu, sunetul emis de fluierul câinilor este recunoscut de câini, dar nu și de oameni. Audio este una dintre cele mai atrăgătoare elemente ale oricărei prezentări multimedia de succes. Impactul sunetului stabilește starea de spirit și ambianța unei prezentări. Acesta poate fi folosit într-o multitudine de moduri într-o aplicație multimedia, de exemplu, pentru a întări un mesaj sau o temă, pentru a stabili starea de spirit sau pentru a atrage interesul sau a alerta publicul. Ca parte a conținutului audio dintr-un sistem multimedia, pot fi folosite: muzică, efecte sonore sau vorbire pentru a realiza obiectivele unei prezentări. Componenta audio care apare într-o prezentare multimedia poate fi de două tipuri: fie este generată de calculator, fie înregistrată și transformată într-un format digital.

Poate cel mai important element al multimedia este elementul grafic. Prezentările multimedia sunt bazate, în principal, pe grafică. Informațiile comunicate prin imagini sunt mai ușor de înțeles și de reținut. Elementele grafice dintr-un sistem multimedia ar putea fi pictogramele (cum ar fi fotografiile) convertite în format digital cu ajutorul scannerelor sau generate pe computer. Ele pot fi plane (sau bidimensionale), cum ar fi fotografiile și picturi, sau pot apărea solide (sau tridimensionale), precum sculpturile și obiectele din jurul nostru. Ele pot fi elemente grafice statice sau animate. În plus, animațiile pot fi bidimensionale, la fel ca în filmele vechi de desene animate sau tridimensionale.

Textul generat de calculator este un alt element al comunicației multimedia. Câteva

cuvinte care apar într-un sistem multimedia predominant grafic pot avea un efect puternic. Pe Internet, textul este utilizat mult mai mult decât pe produsele multimedia de sine stătătoare, deci are o importanță suplimentară. De asemenea, textul și arta pot fi amestecate în moduri interesante pentru a întări mesajul transmis. Textul poate fi, de asemenea, animat în diferite moduri pentru a atrage interesul.

Evident, componenta video și cea audio sunt strâns legate și împreună sunt cele mai eficiente mijloace de comunicare care pot face parte dintr-un sistem multimedia.

1.1. Comunicația audio

Vocea este metoda predominantă prin care oamenii comunică. Suntem atât de obișnuiți să vorbim și să ascultăm încât considerăm acest lucru ceva natural. Dar sunetul există în multe forme diferite și fiecare formă are propriul scop și caracteristici specifice.

Sunetul este interpretarea creierului pentru impulsurile electrice trimise de urechea interioară prin sistemul nervos. Există sunete pe care urechea umană nu le poate percepe. Câinii de exemplu au posibilitatea de a auzi aceste sunete în afara gamei de frecvență disponibilă ființelor umane .

1.1.1. Natura sunetului.

Atunci când un obiect se deplasează înainte și înapoi (vibrează), acesta împinge aerul din imediata apropiere iar când revine în poziția inițială, creează o zonă cu presiune mai slabă. Acest proces de oscilație creează un val similar cu valurile care sunt create atunci când aruncați o piatră în ape liniștite. Aceste unde sonore circulă în toate direcțiile, extinzându-se de la sursa de sunet sub forma unor baloane. Sunetul este astfel o undă mecanică care se propagă prin materie (atât fluide cât și solide). De zici rezultă că sunetul nu se propagă prin vid!

La om, undele sonore sunt captate de pavilionul urechii (urechea externă) și transmise prin canalul auditiv extern al urechii și ajung la timpan (membrana timpanică). Undele sonore sunt astfel transformate în unde de presiune mecanică care acționează asupra timpanului. Vibrațiile timpanului sunt la mamifere transmise prin trei oscioare specializate (ciocan, nicovală și scăriță) către membrana ferestrei ovale în fluidul din urechea internă. Urechea medie comunică cu cavitatea nazală prin trompa lui Eustachio, care are rolul de a egaliza presiunea din cavitatea urechii medii cu mediul înconjurător. Urechea internă cuprinde atât organul de auz (cochilia sau melcul) cât și aparatul vestibular care este organul de echilibru.

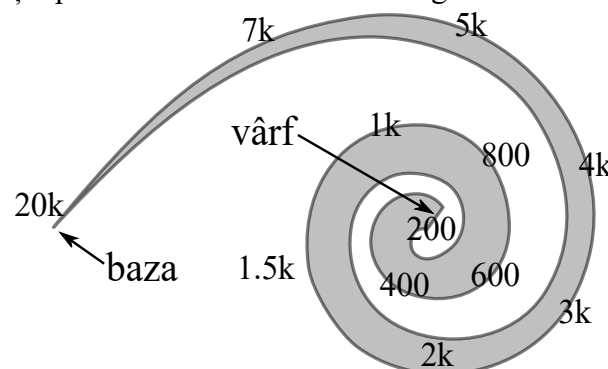


Figura 1.1. Poziția aproximativă a frecvențelor caracteristice a fibrelor nervoase pe membrana bazilară.

Cochilia este o cavitate umplută cu endolimfă, un lichid fluid care primește undele sonore transmise din aer prin urechea externă și medie. Cochilia are o formă spiralată și conține membrana bazilară, a cărei rezonanță este diferită pe lungimea sa. Membrana bazilară separă mecanic componentele de frecvență ale sunetului. Diferitele frecvențe determină maxime locale în amplitudinea undei mecanice care se deplasează în jos pe membrana bazilară în locații diferite

(vezi figura 1.1). Frecvențele înalte (20 kHz) obțin un răspuns maxim la capătul de bază (începutul) cochiliei (locul cel mai apropiat de fereastra ovală). Poate că în mod surprinzător, frecvențele mai joase se deplasează mai departe în membrana bazilară. Cea mai mică frecvență, cu o locație unică (la capătul extrem al cochiliei) este de aproximativ 200 Hz. Astfel, frecvențele sub 200 Hz se bazează în întregime pe interpretarea intervalelor de timp. Pe suprafața membranei bazilare se află un strat celular cunoscut sub numele de organul lui Corti, care este format din celule senzoriale (cili). Organul lui Corti este o structură spirală de-a lungul întregii lungimi a membranei bazilare care transformă mișcarea membranei în impulsuri neurale. Este proiectat astfel încât patru rânduri cili (un rând interior și trei exterioare) să fie perturbate mecanic de trecerea undelor sonore de-a lungul membranei. Fiecare cil se conectează la un neuron care transmite semnalul către sistemul auditiv central (în creier).

Membrana bazilară este structura de susținere pentru aproximativ 12000 de celule senzoriale care formează nervul cohlear. Membrana bazilară este mai rigidă în apropierea ferestrei ovale și devine mai flexibilă spre capătul opus, permițându-i să acționeze ca un analizor spectral de frecvență. Când este expus la un semnal de înaltă frecvență, membrana bazilară rezonază unde este rigidă, rezultând în excitarea celulelor nervoase mai apropiate de fereastra ovală. De asemenea, sunetele de joasă frecvență excită celulele nervoase la capătul îndepărtat al membranei bazilare. Acest lucru face ca fibrele specifice din nervul cohlear să răspundă unor frecvențe specifice.

Celulele nervoase transmit informații generând impulsuri electrice scurte numite potențiale de acțiune. O celulă nervoasă de pe membrana bazilară poate codifica informațiile audio prin producerea unui potențial de acțiune ca răspuns la fiecare ciclu al vibrației. De exemplu, un val de sunet de 200 Hz poate fi reprezentat de un neuron care produce 200 de potențiale de acțiune pe secundă. Această funcționare este posibilă doar pentru frecvențe sub aproximativ 500 Hz, rata maximă la care neuronii pot produce potențiale de acțiune. Urechea umană rezolvă această problemă, utilizând alternativ mai multe celule nervoase în realizarea acestei singure sarcini. De exemplu, un ton sonor de 3000 de hertzi poate fi reprezentat de zece celule nervoase care transmit alternativ impulsuri de 300 de ori pe secundă. Aceasta extinde intervalul în care este folosită această schemă de codificare a informațiilor până la pragul de 4 kHz, deasupra căruia se folosește exclusiv poziția celulelor excitate de-a lungul membranei bazilare.

După cum a fost menționat anterior, numai frecvențele peste 200 Hz au maxime unice pe membrana bazilară. Aceasta înseamnă că fibrele particulare din nervul auditiv sunt maxim active atunci când frecvența lor este prezentă în sunet. Frecvența la care o fibră nervoasă este cea mai activă se numește frecvență caracteristică. Acțiunea mecanică a cochiliei face ca frecvențele sub aproximativ 4000 Hz să fie transmise direct ca forme de undă în nervul auditiv. Adică, pentru frecvențe mai joase, membrana bazilară acționează mai degrabă ca un microfon care traduce valurile de presiune mecanice în valuri de activare neuronală în timp. Aceasta înseamnă că, între aproximativ 200 Hz și 4 kHz, există atât o codare pozițională (unde activitatea în fibre specifice indică componentele de frecvență din sunet), cât și un codare temporală (unde modelul temporal al undelor sonore apare ca un model temporal de activare a fibrelor). Sub 200 Hz, există doar o reprezentare temporală a unui semnal, adică urechea nu face analiză spectrală, deci funcționează doar ca un microfon pentru a transforma modelul mecanic într-unul neural.

1.1.2. Proprietăți fizice ale sunetului

Undele sonore sunt perturbări ale aerului (sau ale altor medii de transmisie). Unda constă în comprimări și rarefierii longitudinale ale aerului. Astfel, toate undele pot fi reprezentate ca o formă de undă standard care descrie compresiile și rarefierile. Compresiile corespund „văilor” iar rarefierile „vârfurilor” în Figura 1.2, care prezintă o formă de undă tipică. O formă de undă oferă o măsură a vitezei particulelor de aer și a distanței pe care o parcurge pentru un sunet dat într-un mediu dat. Amplitudinea măsoară intensitatea relativă a sunetului, care este distanța

dintre o „vale” și un „vârf”, așa cum se arată în Figura 1.2. Amplitudinea determină volumul sunetului. Unitatea de măsură a volumului este un decibelul (dB).

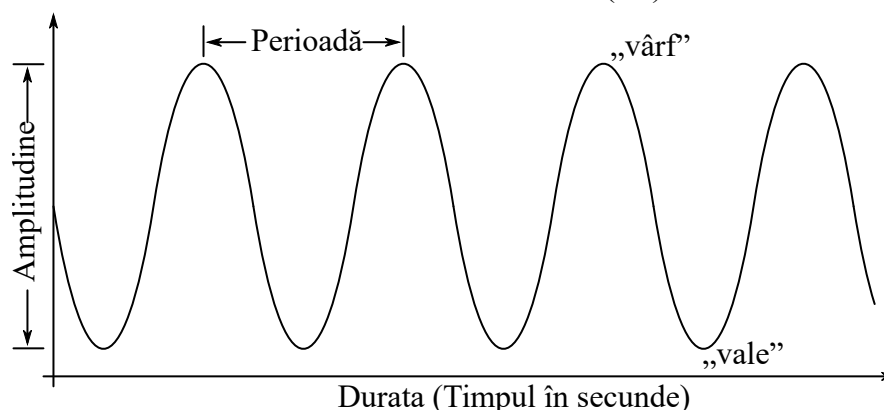


Figura 1.2. Caracteristicile unei forme de undă.

1.1.2.1. Frecvența

Diferența de timp dintre formarea a două „vârfuri” este denumită perioadă. Se măsoară în secunde (vezi figura 1.2). Un anumit număr de creștături (vârfuri) pot apărea într-o secundă. Numărul de vârfuri care apar într-o secundă reprezintă frecvența. Un alt termen asociat cu frecvența este pasul. Dacă un obiect oscilează rapid, acesta creează un sunet „înalt”. Un sunet cu frecvență joasă pe de altă parte este produs de un obiect care vibrează lent, cum ar fi corzile mai groase ale unui pian sau chitară. Frecvența este măsurată prin numărul de cicluri (vibrații) pe secundă, iar unitatea de frecvență este hertz (Hz). Frecvența poate fi de asemenea definită ca numărul de unde care trec printr-un punct într-o secundă.

Gama de frecvențe general acceptată pentru urechea umană este de la 20 Hz până la 20000 Hz (sau 20 kHz)¹ [6]. În condiții ideale de laborator, oamenii pot auzi sunete începând de la 12 Hz și până la 28 kHz. Cu toate acestea, oamenii sunt cei mai sensibili la frecvențele (adică capabile să discearnă la cea mai mică intensitate) între 2 kHz și 5 kHz.

Sunetele sub 20 Hz se numesc infrasunete, în timp ce cele peste 20 kHz sunt numite ultrasunete.

1.1.2.2. Lungimea de undă

Lungimea de undă este de obicei reprezentată distanța dintre punctele corespunzătoare consecutive ale aceleiași faze, cum ar fi punctul de vârf maxim sau minim, sau trecerile prin zero. Lungimea de undă este notată în general cu litera greacă lambda (λ). Lungimea de undă se măsoară în metri sau submultipli ai acestuia.

De exemplu, pentru o undă sinusoidală care se deplasează la o viteză de undă fixă, lungimea de undă este invers proporțională cu frecvența (unde cu frecvențe mai mari au lungimi de undă mai scurte, iar frecvențele mai mici au lungimi de undă mai lungi).

1.1.2.3. Efectul Doppler

Efectul Doppler constă în variația frecvenței unei unde emise de o sursă de oscilații, dacă aceasta se află în mișcare față de receptor. Frecvența măsurată crește atunci când sursa se apropie de receptor și scade când sursa se depărtează de receptor.

Acesta este motivul pentru care sunetul claxonului unei mașini care se apropie sună mai ascutit (frecvențe mai înalte) când se apropie de tine, dar brusc devine grav (frecvențe mai joase) atunci când se îndepărtează.

¹ gama corespunde undelor sonore propagate prin aer la 20 °C, cu lungimi de undă de la 17 m până la 1.7 cm

1.1.2.4. Banda de frecvențe

Banda de frecvențe este definită ca diferența dintre frecvența cea mai înaltă și cea mai joasă conținută de un semnal. Astfel un pian standard, pentru care cea mai joasă notă are frecvența de 27.5 Hz iar cea mai înaltă notă are frecvența de 4.186 kHz va avea o bandă de frecvențe de 4158.5 Hz (4.186 Hz – 27.5 Hz).

Diferitele animale aud diferite intervale de frecvență – după cum se poate observa în Figura 1.3. Unele animale pot auzi frecvențe mult mai mari decât oamenii, în timp ce altele nu pot auzi sunete la fel de scăzute ca noi. S-a observat că există o corelație între dimensiunea fizică și intervalul de frecvență audibil.

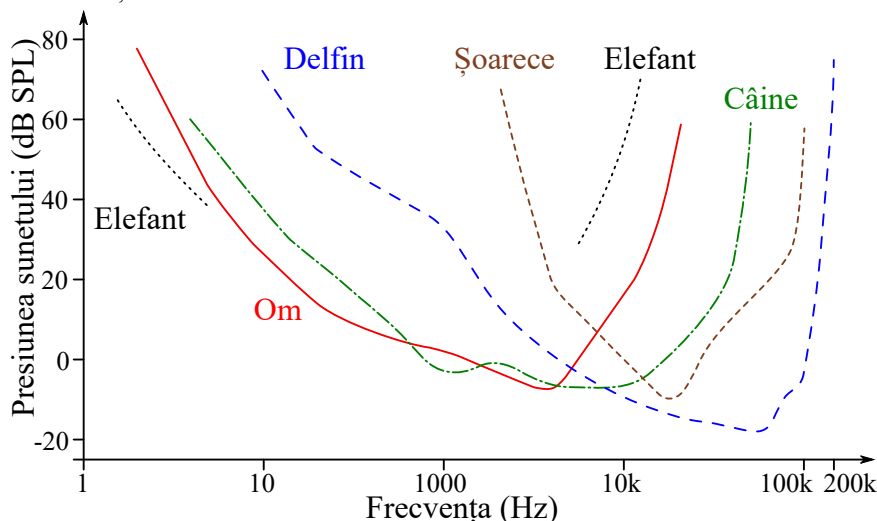


Figura 1.3. Gama de frecvențe audibile pentru diferite animale.

1.1.2.5. Armonici

Puține obiecte produc sunet de o singură frecvență. Cele mai multe instrumente muzicale, de exemplu, generează frecvențe multiple pentru fiecare notă. Acesta este într-adevăr modul în care se poate face diferența dintre instrumente muzicale, de exemplu, o vioară și un flaut, chiar dacă ambele produc note de același frecvență. Combinațiile de frecvențe generate de un instrument sunt cunoscute sub denumirea de *timbru*. Timbrul constă într-o frecvență fundamentală sau principală și alte frecvențe minore cunoscute sub numele de tonuri sau armonici (vezi Figura 1.4).

Sunetele pe care omul le aude de la obiectele vibratoare sunt complexe în sensul că conțin multe frecvențe diferite. Acest lucru se datorează modului în care obiectele vibrează.

O „notă” muzicală generată de un pian diferită de aceeași „notă” generată de un saxofon. În ambele cazuri, „notele” conțin frecvențe diferite suprapuse frecvenței fundamentale comune. Aceste frecvențe diferite, împreună cu diferența dintre timbre, permit distincția între diferitele instrumente.

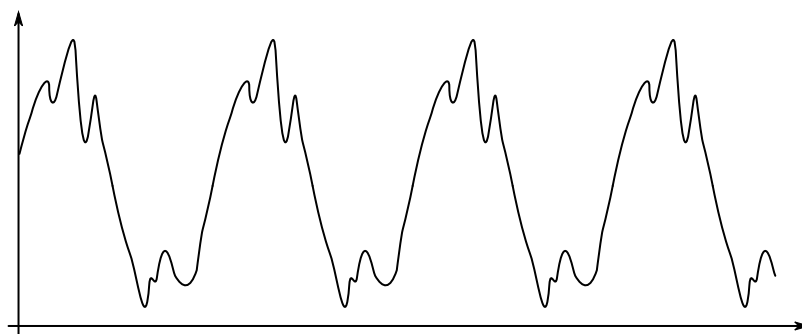


Figura 1.4. Undă sonoră complexă.

1.1.2.6. Viteza sunetului

Viteza de sunet poate determinată prin măsurarea directă a timpului necesar ca undele sonore să parcurgă o anumită distanță. Viteza de propagare variază foarte mult în funcție mediul prin care călătorește. Viteza sunetului crește pe măsură ce temperatura crește și scade odată cu scăderea temperaturii. Tabelul 1.1, de mai jos, oferă valorile pentru viteza sunetului la 0 °C în diferite medii de propagare.

Tabelul 1.1. Viteza sunetului în diferite medii.

Mediu	Viteza (m/s)
Aer	331.5
Hidrogen	1270
Dioxid de carbon	258
Apă	1450
Fier	5100
Sticlă	5500

1.1.2.7. Intensitatea sunetului

Intensitatea sunetului I ca cantitate de energie sonoră a unei unde acustice reprezintă energia acustică transportată de undă, în unitatea de timp, prin unitatea de suprafață perpendiculară pe direcția de propagare. Intensitatea pragului normal de audibilitate este $10^{-12} \text{ W/m}^2 = 1 \text{ pW/m}^2$, iar pragul dureros se află în jurul valorii de 1 W/m^2 .

1.1.3. Parametri psiho-acustici

Psiho-fizica este studiul relației dintre proprietățile unui stimul fizic și răspunsurile senzoriale ale sistemului nervos uman, așa cum au fost raportate de subiecți în situații experimentale. În mod natural, psiho-fizica auditivă (psiho-acustica) este în mod special preocupată de auz. Auzul este percepția sunetului. Percepția este definită în mod obișnuit ca fiind conștientizarea prin simțuri, o definiție circulară tipică a proceselor de nivel superior în organisme vii.

Urechea umană are o gamă și sensibilitate extraordinară. Aceasta ne poate oferi o multitudine de senzații subiective cum ar fi: înălțimea, tăria, timbru, direcția, distanța, etc. Pe baza acestora se poate analiza „calitatea” sunetului, nuanțe ale vocii, etc. În Tabelul 1.2 putem observa dependența anumitor percepții ale auzului uman față de mărimi fizice.

Tabelul 1.2. Dependențele unor percepții sonore față de mărimi fizice.

Percepția	Mărimea fizică
Înălțime	Intensitate
Tărie	Intensitate și frecvență
Timbru	Numărul și intensitatea relativă a frecvențelor multiple (efect non-liniar)
Notă	Unitate muzicală de bază cu nume specifice, combinate pentru a genera melodii
Ton	Numărul și intensitatea relativă a frecvențelor multiple

A se evita utilizarea termenilor psiho-acustici *tarie* și *volum sonor*. Această senzație de sunet subiectivă nu este clar măsurabilă fără ambiguități. Termenul „*tarie*” sau „*volum*” este o problemă deoarece aparține psiho-acusticii și acest sentiment personal nu poate fi definit corect (obiectiv). Tonalitatea ca corelare psihologică a forței fizice (amplitudine) este de asemenea afectată de alți parametri în afară de presiunea acustică, incluzând frecvența, lărgimea de bandă, compoziția spectrală, conținutul informației, structura în timp și durata de expunere a semnalului sonor. Același sunet nu va genera aceeași percepție pentru toți indivizii (oameni).

1.1.3.1. Înălțimea sunetului

Înălțimea sunetului este o dimensiune psihologică care variază în funcție de frecvența fizică. Relația dintre pasul perceput și frecvența este aproximativ liniară de la frecvența de 20 Hz până la aproximativ 1000 Hz. Astfel, percepția schimbării cantitative dintre 200 Hz și 300 Hz este aproximativ aceeași cu schimbarea de la 800 Hz la 900 Hz (ambele fiind o diferență de 100 Hz), dar între 1 kHz și 20 kHz, relația este logaritmică. Diferența dintre 1000 Hz și 1100 Hz (o diferență de 10%) este aproximativ aceeași cu diferența dintre 5000 Hz și 5500 Hz (de asemenea 10%, deși o schimbare de 5 ori mai mare în Hz liniar). Acesta este, de asemenea, aceeași cu 12000 Hz și 13200 (10%). Astfel, prin dublarea unui ton de 100 Hz la 200 Hz se obține o schimbare de octavă și schimbarea este foarte distinctivă, dar o creștere a frecvenței cu 100 Hz începând cu 15 kHz, deoarece reprezintă o creștere de aproximativ 1%, este aproape imposibil de detectat. Există mai multe scale tehnice care reprezintă înălțimea sunetului, cum ar fi scara *mel* și scara *bark* (care sunt foarte similare, deși definite în moduri diferite), unde diferențele egale reprezintă cantități egale de schimbare a înălțimii sunetului [7]. Figura 1.5 arată meli pe scara verticală reprezentată față de două scale diferite de frecvență. Linia albastră solidă este etichetată în partea de sus a graficului în hertzi liniari iar linia roșie punctată este etichetată în partea de jos în hertzi logaritmici [7].

Semnificația acestui grafic este că numai frecvențele joase sunt interpretate corect. La frecvențe înalte, senzațiile de înălțime a sunetului sunt strânse împreună și doar diferențe mari sunt vizibile. Se poate observa, de asemenea, că octavele muzicale reprezintă creșteri logaritmice cu valoare constantă: creșterea înălțimii cu o octavă înseamnă dublarea frecvenței. Atât unitatea *bark*, cât și *mel* se interpretează în termeni de distanță liniară de-a lungul membranei bazilare. Adică senzațiile generate de diferența de înălțime a sunetului sunt de fapt datorate distanței de-a lungul membranei bazilare. Adică percepții egale ale înălțimii sunetului sunt percepute la fel datorită distanțe egale între celulele cililor corespunzători de-a lungul membranei baziliare.

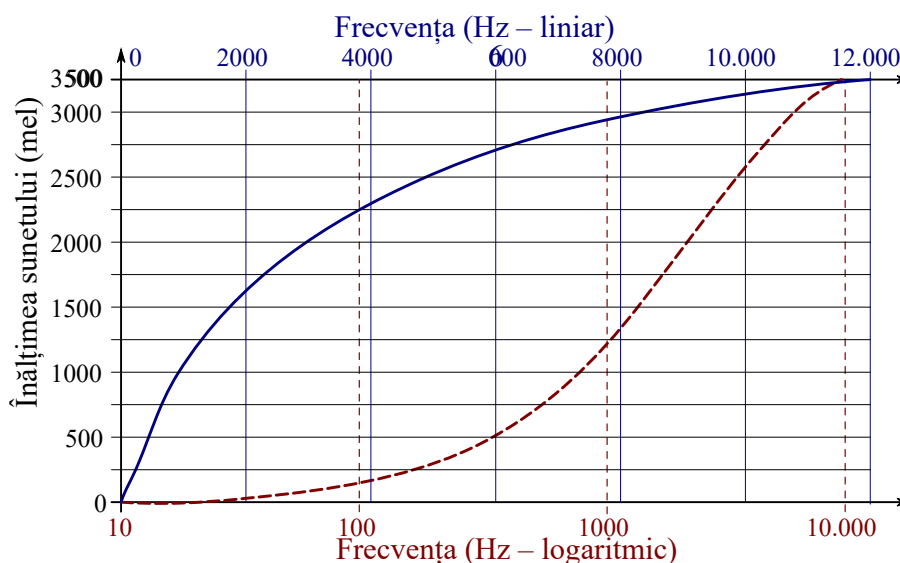


Figura 1.5. Înălțimea sunetului versus frecvența.

1.1.3.2. Nivelul sunetului

Nivelul sunetului este o dimensiune psihologică strâns legată de intensitatea fizică. În Figura 1.6 este prezentat un grafic comparativ pentru senzații de nivel egal al sunetului în planul de frecvență (afișat ca o scară logaritmică) față de intensitate (nivelul presiunii acustice sau dB SPL), o măsură a energiei fizice în sunet. Numerele din grafic reprezintă "nivelele de intensitate a vocii" obținute prin redarea tonurilor pure în diferite puncte și solicitând subiecților să ajusteze un ton de 1000 Hz pentru a avea aceeași tărie sonoră percepută. Se observă că pentru ambele frecvențe sub aproximativ 300 Hz și frecvențe de peste 5 kHz este necesară o intensitate crescută pentru a avea aceeași intensitate ca și tonul de 1000 Hz [7].

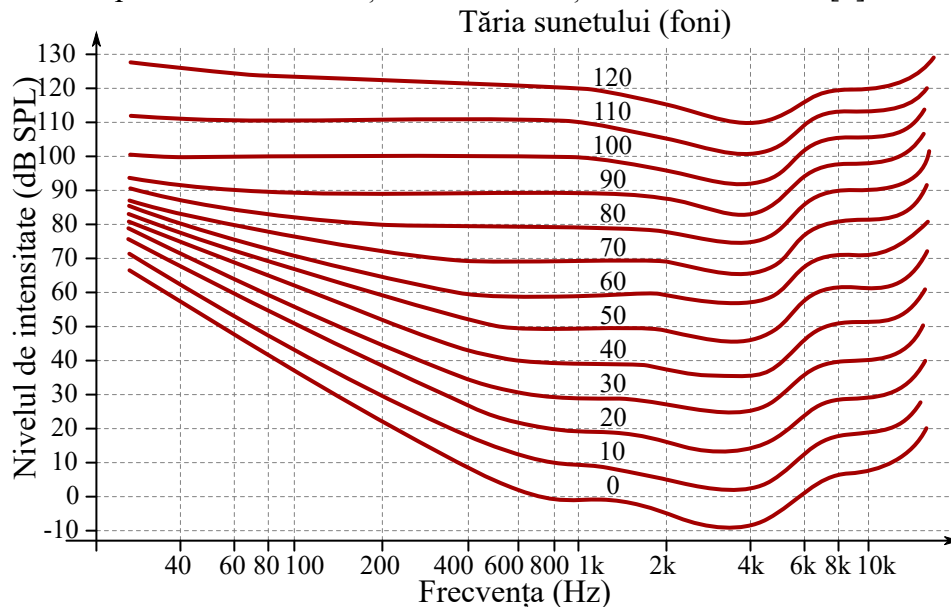


Figura 1.6. Tăria sunetului față de nivelul de intensitate al sunetului.

O proprietate surprinzătoare a sistemului auditiv este reprezentarea neurală a diferențelor de intensitate, adică reprezentarea neurală a domeniului dinamic al sunetului. Auzul uman are o gamă dinamică de peste 100 dB între abia detectabil și nivelul de disconfort. Dar celulele neuronale individuale au doar o gamă dinamică foarte mică de niveluri de activare între lipsa semnalului și rata maximă de activare a celulei. Deci, sistemul auditiv are o varietate de trucuri, unele destul de complexe, pentru a atinge intervalul dinamic de 100 dB.

Ca parametri psiho-acustici pentru a descrie „tăria” sunetului, există „nivelul de intensitate” cu unitatea de măsură fonul și „intensitatea” măsurată în soni.

Valorile sunt mediate și pot diferi cu aproximativ ± 10 dB. Prin presiunea acustică p se înțelege întotdeauna valoarea medie pătratică (RMS – engl. Root-Mean-Square) a presiunii acustice. Amplitudinea presiunii acustice reprezintă valoarea maximă. Presiunea acustică (RMS) este cea mai importantă cantitate din măsurarea sunetului. Urechea umană este un receptor de presiune acustică sau un senzor de presiune acustică, deoarece timpanele (membranele timpanice) sunt deplasate de presiunea acustică ca o cantitate a câmpului sonor. Acesta este efectul important al presiunii acustice în exces. Urechea umană nu este un receptor de energie! Sunetul perceput constă în vibrații de presiune acustică periodice (presiune acustică) care sunt suprapuse peste presiunea atmosferică statică de 101325 Pa (presiunea atmosferică medie) [8].

În Tabelul 1.3 se pot observa valorile pentru diferite surse de zgomot pentru nivelul sonor măsurat în dB SPL și valorile corespunzătoare pentru presiunea și intensitatea sunetului.

Tabelul 1.3. Exemple de valori ale nivelului sonor.

Sursa sonoră (distanța față de sursă)	Nivelul de presiune sonoră (dB SPL)	Presiunea sunetului p (Pa=N/m ²)	Intensitatea sunetului W/m ²
Avion cu reacție (50 m)	140	200	100
Limita de durere	130	63.2	10
Limita de disconfort	120	20	1
Drujbă (1 m)	110	6.3	0.1
Club (1 m de boxe)	100	2	0.01
Camion (10 m)	90	0.63	0.001
Trafic rutier (5 m)	80	0.2	0.000 1
Aspirator (1 m)	70	0.063	0.000 01
Vorbire (1 m)	60	0.02	0.000 001
Locuință	50	0.006 3	0.000 000 1
Biblioteca	40	0.002	0.000 000 01
Dormitor noaptea	30	0.000 63	0.000 000 001
Studio TV	20	0.000 2	0.000 000 000 1
Foșnetul frunzelor	10	0.000 063	0.000 000 000 01
Limita audibilului	0	0.000 02	0.000 000 000 001

1.1.3.3. Localizarea sunetului

Localizarea sunetului este capacitatea de a identifica locația sau originea unui sunet detectat ca direcție și distanță.

Abilitatea de a percepe direcția sunetului funcționează printr-un proces cunoscut sub numele de auz binaural, care în esență înseamnă „auzul cu două urechi”. Pe parcursul evoluției, s-a dovedit că acesta a fost cel mai eficient sistem pentru a permite animalelor să evalueze direcția sunetelor în mediul lor natural. Termenul „binaural” a fost introdus în 1859 pentru a semnifica modalitatea de a asculta același sunet prin ambele urechi sau a două sunete discrete, câte una prin fiecare ureche. Abia în 1916, Carl Stumpf (1848-1936), un filosof și psiholog german, distinge între ascultările dichotice, care se referă la stimularea fiecărei urechi cu un stimul diferit și la ascultarea diotică, stimularea simultană a ambelor urechi cu același stimul [9].

Secretul auzului binaural constă în localizarea urechilor. Deoarece acestea sunt situate pe părțile opuse ale capului, sunetele recepționate de fiecare ureche vor varia în funcție de timpul, volumul și echilibrul frecvențelor. Aceste diferențe sunt indicii pe care creierul le folosește pentru a determina locația sursei sunetului.

Identificarea direcției în plan orizontal se realizează la oameni prin evaluarea diferențelor de timp interaural (fază) pentru frecvențele mai mici și evaluarea diferențelor de nivel interaural pentru frecvențe mai mari. Pentru frecvențe mai mici de 800 Hz, dimensiunile capului (distanța medie între urechi este de 21,5 cm, corespunzătoare unei întârzieri de 625 μ s) sunt mai mici decât jumătatea lungimii de undă a undelor sonore, deci, sistemul auditiv poate determina diferențele de fază între sunetele recepționate de ambele urechi fără confuzie. Pentru frecvențele de peste 1600 Hz, dimensiunile capului sunt mai mari decât lungimea de undă a sunetelor. O determinare neechivocă a direcției de intrare bazată numai pe faza interactivă nu este posibilă la

aceste frecvențe. Cu toate acestea, diferențele de nivel interactiv devin mai mari, iar aceste diferențe de nivel sunt evaluate de către sistemul auditiv.

Pentru localizarea sunetului în planul median (înălțimea sursei sunetului) pot fi utilizați doi detectoare care sunt poziționați la înălțimi diferite. Cu toate acestea, la animale, informația privind elevația este obținută doar prin înclinarea capului, cu condiția ca sunetul să dureze suficient timp pentru a finaliza mișcarea. Acest lucru explică comportamentul înnăscut al câinilor, de exemplu, de a înclina capul într-o parte atunci când aceștia încearcă să localizeze exact un sunet.

Capitolul 2. CODEC-uri video

2.1. Scurtă istorie a CODEC-urilor video

Tehnicile de compresie video digitale au jucat un rol important în domeniul telecomunicațiilor și al sistemelor multimedia unde lățimea de bandă este o resursă limitată. Prin urmare, tehnicile de codare video sunt indispensabile în vederea reducerii cantității de informații necesare pentru stocarea și transmiterea unei secvențe de imagini, fără a pierde o calitate a acesteia, din punct de vedere a utilizatorului uman. Tehnicile moderne de compresie implică circuite electronice complexe, iar costul acestora poate fi menținut la un nivel acceptabil datorită evoluției continue a tehnologiei de calcul digitale. Standardizarea tehnicilor de compresie video este, prin urmare, esențială [10].

Metoda de codificare directă a informației prin modulația impulsurilor (PCM¹) pentru semnale TV a fost introdusă în anii 1970 și necesită o lățime de bandă de 140 Mbit/s. Metoda se bazează pe experiența dobândită în special în domeniul telefoniei digitale multicanal, dar rata transfer a datelor necesară a limitat aplicarea sa doar la distribuția programelor de televiziune și de editare în studio. Aplicarea acesteia în televiziunea digitală prin satelit a devenit atractivă deoarece semnalele erau compatibile cu sistemul de acces cu partajare a timpului. Sisteme experimentale au fost utilizate în anii 1980 și necesitau rate de transfer de aproximativ 45 Mbit/s pentru semnale NTSC și 60 Mbit/s pentru semnale PAL².

Spre sfârșitul anilor 1980, dintre cele 15 propuneri de algoritmi pentru videoconferințe pe bază de blocuri care au fost supuse comisiei de standardizare a telecomunicațiilor al Uniunii Internaționale a Telecomunicațiilor (ITU-T), 14 s-au bazat pe transformata cosinus discretă (DCT – engl. *Discrete Cosine Transform*) și doar una pe cuantificarea vectorilor (VQ – engl. *vector quantisation*). Calitatea subiectivă a secvențelor video prezentate arăta că nu există diferențe semnificative între cele două tehnici de codificare. În paralel cu investigația ITU-T în perioada 1984–88, grupul Joint Photographic Experts Group (JPEG) a fost interesat și de compresia imaginilor statice. Ei au ales DCT ca unitate principală de comprimare, în principal din cauza posibilității de transmitere progresivă a imaginii. Decizia JPEG a influențat, fără îndoială, ITU-T în favoarea DCT față de VQ. La acel moment exista deja implementări ale DCT în microprocesoare și pe procesoarele de semnal digital (DSP – engl. *digital signal processor*).

La sfârșitul anilor 1980, era clar că CODEC-ul ITU-T recomandat pentru videoconferință ar folosi o combinație de DPCM interframe pentru o întârziere minimă de codare și DCT. CODEC-ul oferea o calitate îmbunătățită a imaginii față de standardul H.120. De fapt, calitatea imaginii pentru aplicațiile de videoconferință a fost considerată satisfăcătoare pentru rate de 384 kbit/s sau mai mare, și o calitate bună era posibilă la rate semnificativ mai mari de aproximativ 1 Mbit/s. Acest efort, deși inițial a fost direcționat spre codarea video la 384 kbit/s, a fost ulterior extins la sisteme bazate pe multipli de 64 kbit/s ($p \times 64$ kbits, unde p poate avea valori de la 1 la 30). Definiția standard a fost finalizată la sfârșitul anului 1989 și este denumită oficial standardul H.261 (metoda de codare este adesea menționată ca „ $p \times 64$ ”).

Succesul standardului H.261 a reprezentat o piatră de hotar pentru codarea la rate de compresie mici la o calitate video rezonabilă. La începutul anilor 1990, Motion Picture Experts Group (MPEG) a început investigarea tehnicilor de codificare pentru stocarea video, cum ar fi CD-ROM-urile. Scopul a fost de a dezvolta un CODEC video capabil de a comprima un film

1 PCM – din engl. *Pulse Code Modulation* – este o metodă de codificare a informației într-un semnal digital prin schimbarea amplitudinii impulsurilor. Amplitudinea impulsurilor este limitată la câteva valori predefinite și astfel semnalul este mai puțin sensibil la zgomot.

2 PAL – din engl. *Phase Alternate Line* – este un sistem analogic de codificare al semnalelor de televiziune color apărut în Europa de Vest și folosit în mai multe țări europene sau ne-europene ce folosește folosește 625 linii (din care doar 575 sunt vizibile) la o frecvență de afișare de 25 cadre pe secundă (sau 50 întrețesute).

extrem de activ, cum ar fi filme, pe hard-discuri, cu performanțe comparabile cu cele ale sistemelor video VHS¹ și VCR². De fapt, standardului H.261 a fost folosit ca punct de plecare în designul CODEC-ului. Prima generație de MPEG, numită standardul MPEG-1, a fost capabilă să realizeze această sarcină la 1,5 Mbit/s. Deoarece pentru stocarea video, întârzierile de codare și decodare nu reprezintă o constrângere majoră, se poate întârzia tranzacționarea pentru a eficientiza comprimare. De exemplu, în domeniul temporal, poate fi folosit un DCT mai degrabă decât un DPCM sau poate fi utilizat un DPCM dar cu o compensare mai bună a mișcării, astfel încât compensarea mișcării să elimine corelația temporală. Această din urmă opțiune a fost adoptată în cadrul MPEG-1.

Este ironic faptul că în dezvoltarea lui H.261, compensarea mișcării a fost considerată a fi opțională, deoarece s-a crezut că, după compensarea mișcării, rămânea doar puțină informație pentru a decorela de DCT. Cu toate acestea, cercetările ulterioare au arătat că o compensare eficientă a mișcării poate crește semnificativ rata de compresie. De exemplu, este dificil să se compenseze fundalul descoperit, cu excepția cazului în care cineva privește înaintea mișcării obiectelor. Acesta a fost principiul de bază în MPEG-1, unde mișcarea în majoritatea imaginilor este privită din trecut și din viitor, iar acest lucru s-a dovedit a fi foarte eficient.

Deși MPEG-1 a fost optimizat pentru aplicații care utilizează 25 de cadre pe secundă (în format european) sau 30 de cadre pe secundă (în America de Nord) la rate cuprinse între 1,2–1,5 Mbit/s (pentru o calitate a imaginii comparabilă cu VCR-urile), acesta poate fi cu siguranță utilizat la rate mai mari de biți și rezoluții. Generația următoare numită MPEG-2 a avut un impact semnificativ într-o serie de aplicații, cum ar fi transmisiile digitale terestre, televiziunea digitală prin satelit, televiziunea digitală prin cablu, DVD³-ul și multe altele. În noiembrie 1998, OnDigital din Regatul Unit a început difuzarea programelor terestre de programe BBC și ITV în format MPEG-2, și aproape în același timp, câțiva operatori de satelit, cum ar fi Sky-Digital, au lansat imagini de televiziune codificate MPEG-2 directe către case. De fapt, videoclipul digital datorează o mare parte din succesul său standardului MPEG-2.

Deoarece în MPEG-2 numărul imaginilor prezise bidirecțional este la discreția algoritmului de codare, acest număr poate fi ales pentru o întârziere de codare acceptabilă. Această tehnică poate fi utilizată apoi pentru sistemele de telecomunicații. Din acest motiv, ITU-T a adoptat, de asemenea, standardului MPEG-2 sub numele generic al H.262 pentru telecomunicații. Urmând standardul MPEG-2, codarea televiziunii de înaltă definiție (HDTV) a fost considerată a fi următoarea cerință. Acest proiect a devenit cunoscut sub numele de MPEG-3. Cu toate acestea, versatilitatea MPEG-2, fiind capabilă să codifice orice rezoluție, nu a lăsat niciun loc pentru MPEG-3 și, prin urmare, a fost abandonat.

După MPEG-3, grupul MPEG a început să lucreze la un codec video cu rate foarte reduse, sub numele de MPEG-4. Înainte de a obține o calitate acceptabilă a imaginii la astfel de rate de biți, au apărut noi cerințe. Acestea au fost cauzate în principal de cerințele multimedia, unde a existat o cerere considerabilă de codificare a scenelor 3D, grafice și sintetice, precum și a scenelor naturale. În mod ironic, criticii spun că, din moment ce MPEG-4 nu a putut livra CODEC-ul cu rate foarte scăzute pe care la promis, au fost schimbate specificațiile.

Un standard cu rate foarte scăzute de biți a fost realizat de ITU-T și un nou CODEC video numit H.263 a fost conceput pentru a îndeplini obiectivul MPEG-4. Acest codec, care este o extensie a lui H.261, dar care utilizează lecțiile învățate de la dezvoltările MPEG, este suficient de sofisticat pentru a codifica imagini video cu dimensiuni reduse la rate de cadre scăzute în intervalul 10–64 kbit/s. De-a lungul anilor, eficiența de compresie a acestui codec a fost

1 VHS – din engl. *Video Home System* (adică sistemul video de acasă) este un standard de înregistrare video analogic pe bandă magnetică de 1/2 inci, pentru publicul larg, dezvoltat de compania JVC și lansat în 1976.

2 VCR – din engl. *Video-Cassette Recorder* este un dispozitiv electromecanic care înregistrează semnale audio și video analogice pe o videocasetă cu bandă magnetică detașabilă și poate reda înregistrarea.

3 DVD – din engl. *Digital Video Disc* sau *Digital Versatile Disc* este un mediu de stocare optic care vine ca succesor al CD-ului.

îmbunătățită constant prin mai multe iterații și modificări. De-a lungul evoluției sale, CODEC-ul a fost apoi redenumit H.263+ și H.263++ pentru a indica îmbunătățirile. Datorită unei strategii foarte eficiente de codare folosită în acest codec, acesta poate fi utilizat și pentru imagini de înaltă rezoluție, cum ar fi HDTV, deși la rate mai mari de biți.

Succesul CODEC-ului video MPEG-2/H.262 în cadrul efortului comun al MPEG și ITU a încurajat cele două organisme standard să continue colaborarea. În 1997, grupul de experți în codarea video a ITU-T a lansat proiectul de codificare video avansată (AVC), sub numele de H.26L, cu L pentru obiectivele de lungă durată. În 2001, ISO/IEC MPEG s-a alăturat proiectului și a format o echipă comună (JVT – Joint Video Team). Obiectivul JVT a fost acela de a crea un singur standard de codare video care să depășească cele mai optimizate CODEC-uri video H.263 și MPEG-4. Prima versiune a standardului H.26L a fost finalizată din punct de vedere tehnic în anul 2002 și a fost ratificată oficial la sfârșitul anului 2003. În cele din urmă, CODEC-ul H.26L a fost numit H.264 de către comunitatea ITU-U și MPEG-4 partea 10 a grupului ISO/IEC MPEG.

În anii 2010, H.264/MPEG-4 v10 (cunoscut și sub denumirea de AVC – Advanced Video Codec) a devenit un codec video de facto pentru toate aplicațiile. Atât HDTV terestru cât și prin satelit sunt difuzate în principal în H.264. Dispozitivele noi de stocare, cum ar fi Blu-ray de mare capacitate, pentru o calitate sporită și eficiență de stocare, utilizează codec-uri familiale H.264. Noile servicii video lansate de telecomunicații, cum ar fi televiziunea prin Internet Protocol (IPTV), video prin Internet, captura TV și streaming video în general, utilizează familia de CODEC-uri H.264. O variantă a acestui codec a fost produsă sub diferite nume de marcă, cum ar fi Microsoft VC1, Chinese ASV, DivX+, Sorenson etc., care se bazează pe H.264. Astfel H.264 era un codec pentru toate aplicațiile de comunicații video, de distribuție, de stocare și de rețele.

În prezent standardul utilizat este HEVC (High Efficiency Video Coding), cunoscut și ca H.265 și MPEG-H Partea 2, este un standard de comprimare video proiectat ca succesor al AVC. În comparație cu AVC, HEVC oferă o mai bună compresie a datelor cu 25% până la 50%, la același nivel de calitate video sau o calitate îmbunătățită substanțial a videoclipului la aceeași rată de biți. Acesta susține rezoluții de până la 8192×4320, inclusiv 8K UHD, și spre deosebire de AVC în primul rând pe 8 biți, profilul principal Main10 al HEVC este încorporat în aproape toate componentele hardware.

În esență, HEVC este o extensie a conceptelor din H.264/MPEG-4 AVC. Ambele funcționează prin compararea diferitelor părți ale unui cadru video pentru a găsi zone redundante, atât într-un singur cadru, cât și între cadre consecutive. Aceste zone redundante sunt apoi înlocuite cu o scurtă descriere în locul pixelilor originali. Modificările primare pentru HEVC includ extinderea zonei de comparare a modelului și a codurilor diferențiale de la 16×16 pixeli la dimensiuni de până la 64×64, segmentarea îmbunătățită a dimensiunii blocurilor variabile, predicția îmbunătățită „întra” în cadrul aceleiași imagini, predicția vectorilor și gruparea regiunilor de mișcare, o filtrare îmbunătățită a compensării mișcării și o etapă suplimentară de filtrare numită filtrare a offset-ului adaptabilă. Utilizarea eficientă a acestor îmbunătățiri necesită mult mai multă capacitate de procesare a semnalului pentru comprimarea fluxului video, dar are un impact mai mic asupra cantității de calcul necesare pentru decompresie.

Dacă prima versiune a standardului HEVC/H.265 a fost lansată în 2013 ITU-T, ultima versiune disponibilă la momentul de față (2019) este versiunea a 4-a aprobată la 22 decembrie 2016. Sistemele de operare Windows 10, iOS 11, Android Lollipop (5.0) oferă suport nativ pentru HEVC H.265. Procesoarele Nvidia începând cu GTX 970, Intel cu generația Skylake, AMD începând cu Carrizo oferă suport hardware pentru HEVC.

2.2. Principiile compresiei video

Analiza statistică a semnalelor video indică faptul că există o corelație puternică atât între cadrele succesive ale imaginilor, cât și în interiorul elementelor de imagine. Teoretic, de-corelarea acestor semnale poate duce la compresia lățimii de bandă fără a afecta în mod

semnificativ rezoluția imaginii. Mai mult, insensibilitatea sistemului vizual uman la pierderea anumitor informații vizuale spațio-temporale poate fi exploatată pentru a obține o reducere suplimentară. Prin urmare, pot fi utilizate tehnici de compresie cu pierderi subiective pentru a reduce ratele de biți video, menținând în același timp o calitate acceptabilă a imaginii [10].

Pentru codarea imaginilor statice, este exploatată numai corelația spațială. O astfel de tehnică de codare se numește codare *intra-frame* și reprezintă baza pentru codificarea JPEG (Joint Photographic Experts Group). Dacă se folosește și corelația temporală, se numește codificare *inter-frame*. Interfața de codificare predictivă este principalul principiu de codare utilizat în toate CODEC-urile video standard, cum ar fi H.261, H.263, H.264 și MPEG-1, -2 și -4. Se bazează pe următoarele trei principii fundamentale de reducere a redundanței:

1. Reducerea redundanței spațiale: pentru a reduce redundanța spațială între pixeli într-o imagine (similaritatea pixelilor, în cadrul imaginii) prin utilizarea unor compresoare de date, cum ar fi codificarea utilizând funcții de transformare.
2. Reducerea temporară a redundanței: pentru a elimina asemănările dintre imaginile succesive prin codificarea diferențelor.
3. Codificarea entropiei: pentru a reduce redundanța între simbolurile de date comprimate, folosind tehnici de codare cu lungime variabilă (VLC – Variable Length Coding).

2.2.1. Reducerea redundanțelor spațiale

2.2.1.1. Codarea predictivă

La începuturile compresiei imaginilor, atât instrumentele de procesare a semnalelor, cât și dispozitivele de stocare erau limitate. În acel moment, o metodă simplă de reducere a redundanței a fost aceea de a prezice valoarea pixelilor pe baza valorilor codificate anterior și de a codifica eroarea de predicție. Această metodă se numește modularea codurilor de impulsuri diferențiale (DPCM – engl. Differential Pulse Code Modulation). La decodare, semnalul de eroare primit este adăugat predicției pentru a reconstrui semnalul inițial.

Cele mai bune previziuni sunt cele ale pixelilor vecini, fie din același cadru, fie din pixeli din cadrul anterior, sau combinațiile acestora. Primul se numește codificare predictivă *intra-frame* iar celalalt codificarea predictivă *inter-frame*. Combinația lor se numește codificare predictivă hibridă.

Trebuie remarcat faptul că indiferent ce predicție este folosită, fiecare pixel este codificat în mod predictiv. Numărul minim de biți care poate fi atribuit fiecărei erori de predicție este de 1 bit. Prin urmare, acest tip de codare nu este potrivit pentru codarea video cu rate reduse. Ratele de biți mai mici pot fi obținute dacă un grup de pixeli este codificat împreună, astfel încât bitul mediu per pixel poate fi mai mic de 1 bit. Transformarea pe blocuri este cea mai potrivită pentru acest scop. În ciuda acestui fapt, DPCM este încă folosit în compresia video. De exemplu, DPCM *inter-frame* are o latență de codare mai mică decât codarea *inter-frame* a blocurilor. De asemenea, DPCM ar putea fi utilizat în codarea vectorilor de mișcare sau a adreselor bloc. Dacă vectorii de mișcare dintr-un obiect în mișcare se mișcă în aceeași direcție, codarea diferențelor lor va reduce informațiile vectorului de mișcare. Bineînțeles, codarea ar fi lipsită de pierderi.

2.2.1.2. Codarea prin transformare

Codificarea prin transformarea domeniului de reprezentare este folosită în principal pentru a elimina redundanțele spațiale în imagini prin maparea pixelilor într-un domeniu de transformare înainte de reducerea datelor. Avantajul codificării prin transformare în obținerea compresiei datelor este aceea că energia imaginii majorității scenelor naturale este concentrată, în principal, în regiunea de frecvență și, prin urmare, conține doar câțiva coeficienți de transformare. Acești coeficienți pot fi apoi cuantificați cu scopul de a elimina coeficienții

nesemnificativi, fără a afecta semnificativ calitatea imaginii reconstruite. Acest proces de cuantificare este, totuși, cu pierderi prin faptul că valorile inițiale nu pot fi reconstituite.

Pentru a vedea cum codificarea transformării poate duce la comprimarea datelor, se pot considera situația a doi pixeli alăturați, după cum se arată în Figura 2.1.

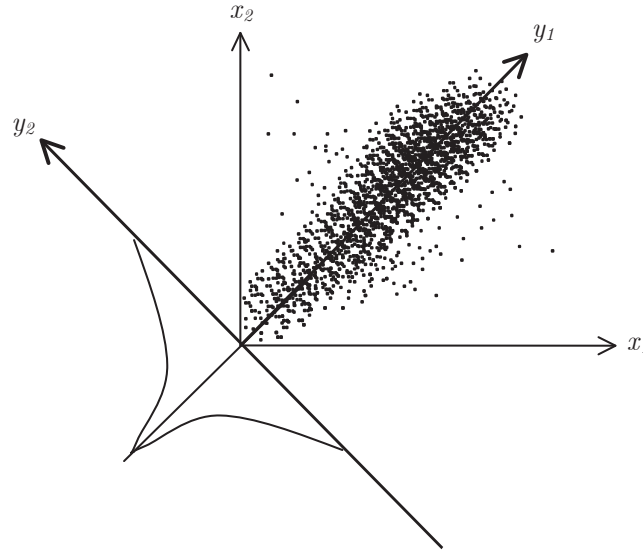


Figura 2.1. Distribuția culorii dintre doi pixeli alăturați.

Deși fiecare pixel x_1 sau x_2 poate lua orice valoare uniformă între 0 (negru) și valoarea maximă 255 (alb), deoarece există o corelație mare (similaritate) între ele, atunci este foarte probabil ca aparițiile lor comune să fie în principal pe o linie de 45 de grade, așa cum se arată în figură. Acum, dacă se rotesc axele de coordonate x_1x_2 cu 45 de grade, într-o nouă poziție y_1y_2 , atunci aparițiile comune ale noilor coordonate au o distribuție uniformă de-a lungul axei y_1 , dar sunt concentrate în jurul valorii de zero pe axa y_2 . Desigur, biții necesari pentru a reprezenta noul parametru y_1 pot fi la fel de mari ca oricare dintre x_1 sau x_2 , dar cele pentru parametrul y_2 sunt mult mai mici. Prin urmare, în medie, y_1 și y_2 pot fi reprezentate la o rată de biți mai mică decât x_1 și x_2 . Rotația coordonatelor x_1x_2 la 45 de grade este o transformare a vectorului $[x_1, x_2]$ printr-o matrice de transformare T , prezentată în ecuația:

$$T = \begin{bmatrix} \cos(45) & \sin(45) \\ \sin(45) & -\cos(45) \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \quad (1)$$

Astfel, în acest exemplu, coeficienții de transformare $[y_1, y_2]$ devin:

$$[y_1, y_2] = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \text{ sau } y_1 = \frac{1}{\sqrt{2}}(x_1 + x_2); \quad y_2 = \frac{1}{\sqrt{2}}(x_1 - x_2) \quad (2)$$

unde y_1 se numește valoarea medie a lui x_1 și x_2 iar y_2 reprezintă diferența reziduală. Factorul de normalizare de $1/\sqrt{2}$ asigură că energia semnalului datorată transformării nu se modifică (teorema lui Parseval). Aceasta înseamnă că energia semnalului în domeniul pixelului, $x_1^2 + x_2^2$ este egală cu energia semnalului din domeniul de transformare $y_1^2 + y_2^2$. Prin urmare, matricea de transformare este ortonormală.

Dacă în loc de 2 pixeli, luăm N pixeli corelați și apoi transformăm coordonatele astfel încât y_1 se află pe diagonala principală a sferei, atunci numai coeficientul y_1 devine semnificativ, iar restul $N-1$ coeficienți, $y_2, y_3, \dots, y_N$, conțin doar informațiile reziduale. Astfel, în comparație cu cazul a 2 pixeli, dimensiunile mai mari ale transformării pot duce la o comprimare mai mare. Exact cât de mari ar trebui să fie dimensiunile depinde de cât de mult pixelii pot fi corelați între ei. De asemenea, elementele matricei de transformare, numite vectori de bază, au un rol important în eficiența compresiei. Acestea ar trebui aleasă astfel încât numai unul dintre

coeficienții de transformare, sau cel mult câțiva dintre ei, să devină semnificativ, iar restul să fie mici. O alegere ideală pentru matricea de transformare este cea care decorelează complet coeficienții de transformare. Astfel, dacă R_{xx} este matricea covarianței pentru o sursă de intrare (pixeli), x , atunci elementele matricei de transformare T sunt calculate astfel încât covarianța coeficienților $R_{yy} = TR_{xx}T^T$ este o matrice diagonală (elementele exceptând diagonala sunt 0). O transformare derivată pe această bază este bine-cunoscuta transformată Karhunen–Loève – KLT (din engl. *Karhunen-Loève Transform*) [11]. Cu toate acestea, deși această transformare este optimă și, prin urmare, poate oferi o eficiență maximă de compresie, nu este adecvată pentru compresia de imagini. Acest lucru se datorează faptului că, pe măsură ce statisticile de imagine se modifică, elementele transformării trebuie recalulate. Astfel, în plus față de complexitatea extra-computațională, aceste elemente trebuie transmise către decodor. Informația suplimentară implicată în transmisie limitează în mod semnificativ eficiența generală a compresiei. Cu toate acestea, KLT este încă utilă și poate fi folosită ca punct de referință pentru evaluarea eficienței de compresie a altor transformări. O alegere mai bună pentru matricea de transformare este aceea a transformării cosinus discrete (DCT). Motivul pentru aceasta este faptul că are vectori de bază bine definiți (fiecși) și vectori de bază lent variabili, care seamănă cu variațiile de intensitate ale celor mai multe scene naturale, astfel încât energia imaginii se potrivește cu câțiva dintre coeficienți. Din acest motiv, performanța sa de distorsionare a ratei este urmată îndeaproape de cea a KLT și are ca rezultat câștigul de compresie aproape identic [11]. La fel de importantă este disponibilitatea unor algoritmi de transformare DCT eficienți care pot fi utilizați, în special în aplicațiile de codare a imaginilor [12].

Deoarece în secvențe de imagini naturale, pixelii sunt corelați atât pe orizontală, cât și pe verticală, precum și în direcția temporală a secvenței de imagini, o alegere naturală pentru DCT este una tridimensională. Cu toate acestea, orice transformare în domeniul temporal necesită stocarea mai multor cadre de imagine, introducând o întârziere destul de lungă, care restricționează aplicarea codării transformate în telecomunicații. Prin urmare, transformarea este limitată la două dimensiuni. Un DCT bidimensional este un proces separabil care este implementat folosind două DCT-uri unidimensionale: una în direcție orizontală urmată de una verticală.

În practică, o astfel de transformare bidimensională se aplică pe un *block* de $8 \times 8 = 64$ de pixeli care are ca rezultat 64 coeficienți de transformare. Alegerea unei astfel de dimensiuni a blocului este un compromis între eficiența de compresie și artefactele de blocare a coeficienților cuantificați în mod grosier. Deși dimensiunile mari ale blocurilor au o eficiență bună de compresie, artefactele de blocare sunt subiectiv foarte deranjante. În stadiul incipient de standardizare a CODEC-urilor video, dimensiunile blocurilor au fost făcute opționale la 4×4 , 8×8 și 16×16 . Dimensiunea blocului în CODEC-urile H.263 este de 8×8 , pentru standardul H.264 acesta poate fi și 4×4 , iar pentru H.265 acesta este de la 16×16 până la 64×64 .

2.2.1.3. Erori de aproximare

Implementarea transformărilor directe și inverse cere ca elementele cosinus să fie approximate cu numere finite. Din cauza acestei aproximații, semnalul reconstruit, chiar și fără o cuantificare, nu poate fi o copie exactă a semnalului de intrare. Pentru aplicațiile de codare a imaginilor și video, această nepotrivire trebuie controlată; în caz contrar, eroarea acumulată datorată aproximării poate să scadă din control, ducând la un artefact în imagine.

O modalitate de a preveni acumularea de erori este de a lăsa eroarea să oscileze între două niveluri mici. Acest lucru garantează că eroarea acumulată nu depășește niciodată limita impusă. Abordarea adoptată în codec-urile standard este de a spune (de exemplu, MPEG-2) că suma tuturor valorilor celor $8 \times 8 = 64$ coeficienți de transformare ar trebui să fie un număr impar (indiferent dacă acestea sunt cuantificate sau nu), la decodare. În cazul în care suma este un număr par, valoarea celui mai mare coeficient de frecvență este fie incrementată, fie decrementată cu 1, în funcție de paritatea valorii coeficientului. Aceasta, desigur, introduce o

eroare foarte mică, dar nu poate fi observată în imagini, din două motive. În primul rând, la transformarea inversă, pixelii reconstruiți sunt împărțiți cu o valoare mare în ordinea lui N^2 . În al doilea rând, deoarece eroarea este introdusă de cel mai înalt coeficient de frecvență, ea apare ca un zgomot foarte mic, de frecvență foarte mică, care nu este deloc perceptibil (ochiul uman este foarte tolerant la zgomotul de frecvență înaltă).

2.2.1.4. Transformata cosinus discretă rapidă

Transformata cosinus discretă (DCT – din engl. *Discrete Cosine Transform*) este utilizată în numeroase aplicații din știință și inginerie, de la compresia cu pierderi a sunetului (de exemplu, standardul MP3) și a imaginilor (de exemplu, standardul JPEG pentru imagini sau standardul MPEG pentru video) (unde pot fi ignorate/eliminate componente mici de frecvență înalte la care ochiul uman este mai puțin sensibil), până la metode spectrale pentru soluționarea numerică a ecuațiilor diferențiale.

Transformata cosinus discretă este similară cu transformata Fourier discretă (DFT – din engl. *Discrete Fourier Transform*) dar care utilizează doar numere reale. Vetterli și Nussbaumer au dovedit că DCT în N puncte poate fi exprimată în termeni de părți reale și imaginare ale unui DFT în N puncte și rotația acestor ieșiri DFT [13]. Harelick a arătat că primii N coeficienți ai unui DFT în $2N$ puncte cu simetria adecvată a valorilor de intrare pot fi utilizați pentru a calcula un DCT în N puncte [14]. Această proprietate este utilizată pentru a construi o structură DCT eficientă și scalabilă.

Transformata cosinus discretă exprimă secvență finită de valori sub forma unei sume de funcții cosinus de frecvențe diferite. Ca aplicabilitate în compresia de date s-a observat că funcția cosinus este mult mai eficientă față de funcția sinus fiind necesare mai puține funcții cosinus pentru a aproxima un anumit semnal.

Există opt variante ale funcției DCT, dintre care patru sunt comune. Cele mai cunoscute forme sunt: DCT-II (de tip II), care este cel mai adesea referită simplu ca transformata cosinus discretă directă DCT sau FDCT (din engl. *Forward Discrete Cosine Transform*) și inversa acesteia DCT-III are este referită ca transformata cosinus discretă inversă IDCT (din engl. *Inverse Discrete Cosine Transform*). Mai este utilizată în compresia de date audio (de exemplu în standardele AAC, MP3) varianta DCT-IV cunoscută sub denumirea de transformata cosinus discretă modificată MDCT (din engl. *Modified Discrete Cosine Transform*).

Transformata cosinus discretă DCT comună, utilizată în compresia de date a fost introdusă în 1974 de către Ahmed, Natarajan și Rao și este dată pentru $u = [1, N-1]$ de următoarea formulă trigonometrică [15]:

$$F(u) = \sqrt{\frac{2}{N}} \sum_{i=0}^{N-1} \left[\Delta(i) \cdot \cos \frac{(2i+1)\pi \cdot u}{2 \cdot N} f(i) \right], \quad \text{unde } \Delta(k) = \begin{cases} \frac{1}{\sqrt{2}}, & \text{pentru } k=0 \\ 1, & \text{în rest} \end{cases} \quad (3)$$

Transformata DCT comună (DCT-II) este adesea utilizată în procesarea de semnal și de imagine, în special pentru compresia de date cu pierderi deoarece are proprietatea de a „compacta energia”: majoritatea energiei semnalului este concentrată în câteva componente de frecvențe scăzute, oferind o alternativă viabilă pentru transformata Karhunen-Loève (KLT) care este transformata optimă pentru a de-corela un semnal dar pentru care nu s-a găsit încă o implementare eficientă.

Variante multidimensionale ale funcțiilor DCT se obțin prin compunerea ecuației (3) pentru fiecare dimensiune în parte. Astfel formula pentru DCT 2D este, cu $\Delta(k)$ definit în (3):

$$F(u, v) = \sqrt{\frac{2}{N}} \sqrt{\frac{2}{M}} \sum_{i=0}^{N-1} \sum_{j=0}^{M-1} \left[\Delta(i) \cdot \Delta(j) \cdot \cos \frac{(2i+1)\pi \cdot u}{2 \cdot N} \cos \frac{(2j+1)\pi \cdot v}{2 \cdot M} f(i, j) \right] \quad (4)$$

Experimentele au arătat că s-ar putea obține puține câștiguri din punct de vedere al

compresiei pentru dimensiuni mari ale blocurilor de date, mai ales când se ia în considerare complexitatea computațională crescută a implementării. Un algoritm DCT rapid va necesita aproximativ dublarea operațiilor aritmetice atunci când numărul de puncte este dublată. Desigur, cea mai bună eficiență a compresiei a fost demonstrată folosind dimensiuni adaptive ale blocurilor la nivel local (de exemplu, 16×16 , 16×8 , 8×8 , 8×4 și 4×4) [16]. În mod inevitabil, aceasta necesită stocarea de informații suplimentare și forțează decodorul să implementeze algoritmi DCT recursivi programabili sau cablați. Dacă dimensiunea DCT devine prea mare, atunci vor apărea mai multe margini (discontinuități locale) rezultând o propagare mai largă a fenomenului Gibbs (artefacte în apropierea tranzițiilor ascuțite dintr-un semnal) și a altor fenomene. În cele din urmă, cu dimensiuni ale blocului de transformare mai mari, termenul DC este și mai sensibil la zgomotul de cuantizare. Astfel compromisul ales pentru standarde precum JPEG și MPEG a fost folosirea blocului de 8×8 de puncte pentru transformata DCT 2D.

Elementele esențiale ale compresiei bazate pe DCT, sunt în esență compresia unui flux de date conținând blocuri 8×8 de mostre de imagini pe fiecare componentă de culoare. Componentele de culoare pot fi comprimate fie în întregime una câte una, fie prin intercalarea alternativă a blocurilor de 8×8 din fiecare componentă pe rând.

La intrarea în codificator, mostrele de imagine sursă sunt grupate în blocuri de 8×8 , deplasate de la numere întregi fără semn cu intervalul $[0, 2^{N-1}]$ la numere întregi cu semn cu intervalul $[-2^{N-1}, 2^{N-1}-1]$ (pentru octeți, $N=8$, deci intervalul valorilor fără semn $[0, 255]$ va fi convertit la intervalul valorilor cu semn pe 8 biți $[-128, 127]$) și care reprezintă intrările către FDCT 2D. O analiză numerică a ecuației FDCT 8×8 arată că, dacă semnalul de intrare compus din 64 (bloc 8×8) valori numerice întregi de N biți, atunci partea nefracțională a numerelor obținute la ieșire (coeficienții DCT) poate crește cu cel mult 3 biți și astfel FDCT va produce valori în intervalul $[-2^{N+2}, 2^{N+2}-1]$. Deci pentru date de intrare reprezentate ca numere cu semn pe 8 biți, din intervalul $[-128, 127]$ coeficienții DCT vor fi în intervalul $[-1024, 1023]$, care pot fi reprezentați ca numere cu semn pe 11 biți). În procesul invers, IDCT generează la ieșire blocuri 8×8 care vor forma imaginea reconstruită. Definițiile matematice ale FDCT 8×8 (5) și IDCT 8×8 (6) sunt date de următoarele ecuații:

$$F(u, v) = \frac{1}{4} \Delta(u) \Delta(v) \left[f(i, j) \sum_{i=0}^8 \sum_{j=0}^8 \cos \frac{(2i+1)\pi u}{16} \cos \frac{(2j+1)\pi v}{16} \right] \quad (5)$$

$$f(u, v) = \frac{1}{4} \left[\Delta(u) \Delta(v) F(i, j) \sum_{i=0}^8 \sum_{j=0}^8 \cos \frac{(2i+1)\pi u}{16} \cos \frac{(2j+1)\pi v}{16} \right] \quad (6)$$

$$\text{unde: } \Delta(k) = \begin{cases} 1/\sqrt{2}, & \text{pentru } k=0 \\ 1, & \text{pentru } k \neq 0 \end{cases}$$

Fiecare bloc de 8×8 de mostre din imaginea sursă constituie de fapt un semnal discret de 64 de puncte care este o funcție a celor două dimensiuni spațiale x și y . FDCT preia un astfel de semnal ca intrare și îl descompune în 64 de semnale de bază ortogonale. Fiecare conține una dintre cele 64 de „frecvențe spațiale” bidimensionale (2D) unice care constituie „spectrul” semnalului de intrare. Ieșirea FDCT este un set de 64 de amplitudini ale semnalului de bază sau „coeficienți DCT” ale căror valori sunt determinate în mod unic de semnalul de intrare de 64 de puncte.

Valorile coeficienților DCT pot fi astfel considerate ca fiind cantitatea relativă a frecvențelor spațiale 2D conținute în semnalul de intrare în 64 de puncte. Coeficientul cu frecvență zero în ambele dimensiuni se numește „coeficient DC”, iar restul de 63 de coeficienți sunt numiți „coeficienți AC”. Deoarece valorile eșantionului variază de obicei lent de la un punct la altul pe o imagine, pasul de procesare FDCT pune la dispoziție o metodă de realizare a compresiei datelor prin concentrarea majorității semnalului în frecvențele spațiale inferioare.

Pentru un bloc de probă de dimensiune 8×8 dintr-o imagine sursă tipică, majoritatea frecvențelor spațiale au amplitudine zero sau aproape de zero și nu trebuie să fie codificate.

La decodare, IDCT inversează această etapă de procesare. Acesta ia cei 64 de coeficienți DCT (care în acel moment au fost cuantificați) și reconstruiește un semnal de imagine de ieșire în 64 de puncte prin însumarea semnalelor de bază. Din punct de vedere matematic, DCT este o mapare unu-la-unu pentru vectori cu 64 de puncte între imagine și domeniul frecvență. Dacă FDCT și IDCT ar putea fi calculate cu acuratețe perfectă și dacă coeficienții DCT nu ar fi cuantificați, semnalul original de 64 de puncte ar putea fi recuperat exact. În principiu, DCT nu introduce nici o pierdere la mostrele de imagine sursă; pur și simplu le transformă într-un domeniu în care pot fi codificate mai eficient.

Pentru a calcula coeficienții de transformare, fiecare transformare unidimensională directă sau inversă necesită opt multiplicări și șapte adunări. Acest proces este repetat pentru cei 64 de coeficienți în ambele direcții: pe orizontală și pe verticală. Astfel metode de reducere a unei astfel de sarcini computaționale enorme sună foarte atrăgător. Faptul că DCT este un tip de transformare Fourier discretă, cu avantajul că toți coeficienții sunt reali, permite folosirea unei transformări rapide, asemănătoare transformării rapide Fourier, pentru a calcula coeficienții de transformare cu complexitate proporțională cu $N \log_2 N$, mai degrabă decât N^2 .

Primul algoritm pentru o DCT rapidă a fost publicat în septembrie 1977 de către Chen, Harrison Smith și Fralick sub forma unui algoritm ușor de implementat prezentat sub forma unei diagrame *Butterfly* (fluturi în engl.) în care operațiile ce execută în paralel sunt ușor de urmărit [12]. În Figura 2.2 se poate observa implementarea propusă pentru varianta rapidă în 8 puncte a DCT. Nodurile intermediare împart o parte din povara computațională, și prin urmare duc la reducerea complexității algoritmului pe ansamblu. În figură, f_{0-7} reprezintă intrările pentru DCT directă (FDCT) și F_{0-7} sunt coeficienții de transformare. Intrările pot fi: fie 8 pixeli ai imaginii sursă, fie opt coeficienți de transformare ai primei etape ai unei transformări bi-dimensionale. În mod similar, pentru transformarea inversă, F_{0-7} reprezintă intrările pentru DCT inversă (IDCT), iar f_{0-7} constituie ieșirile. În algoritmul inițial propus de Chen et al., DCT unidimensională (1D) pe 8 puncte necesită multiplicări cu numere de forma $c_i = \cos(i \cdot \pi / 16)$ sau $s_i = \sin(i \cdot \pi / 16)$ (specificate în interiorul chenarelor) și care pentru a putea fi mai ușor de urmărit valorile negative sunt colorate cu roșu și cele pozitive cu albastru. Aceste constante impun dificultăți în implementare datorită reprezentării interne a datelor deoarece acestea sunt valori iraționale. Implementările DCT ce utilizează virgulă fixă utilizează aproximări ale acestor valori ceea ce introduce erori ce se vor propaga pe parcursul aplicării calculelor.

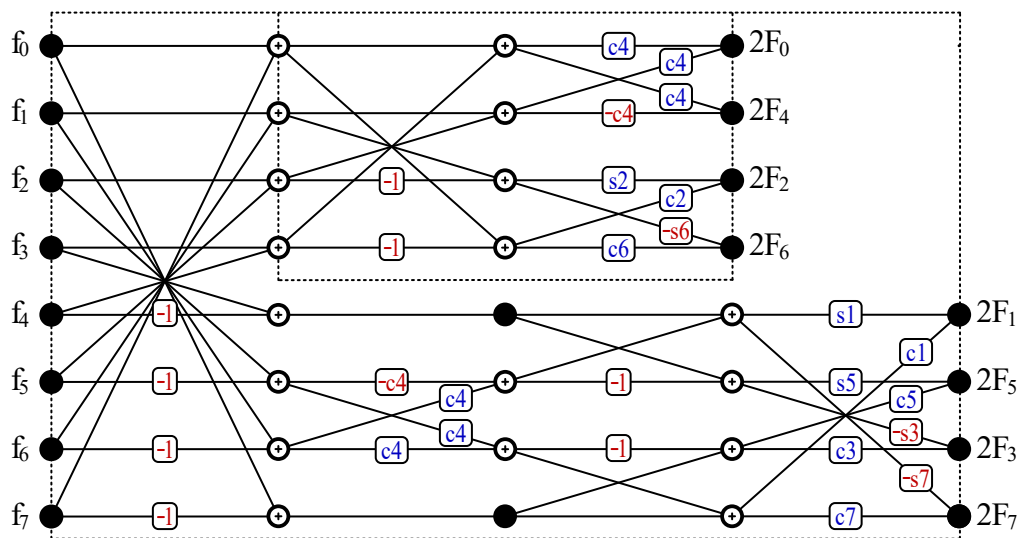


Figura 2.2. Diagrama algoritmului inițial al lui Chen pentru DCT rapidă.

Ținând cont de identitatea trigonometrică $\sin(x) = \cos((\pi/2) - x)$, în figura 2.2 valorile ponderilor de tip sinus pot fi înlocuite cu echivalentele lor cosinus ($s2=c8$, $s3=c5$, $s5=c3$, $s6=c2$, $s7=c1$) și se obține structura din Figura 2.3, care utilizează șapte (7) valori precalculate de la $c1$ la $c7$.

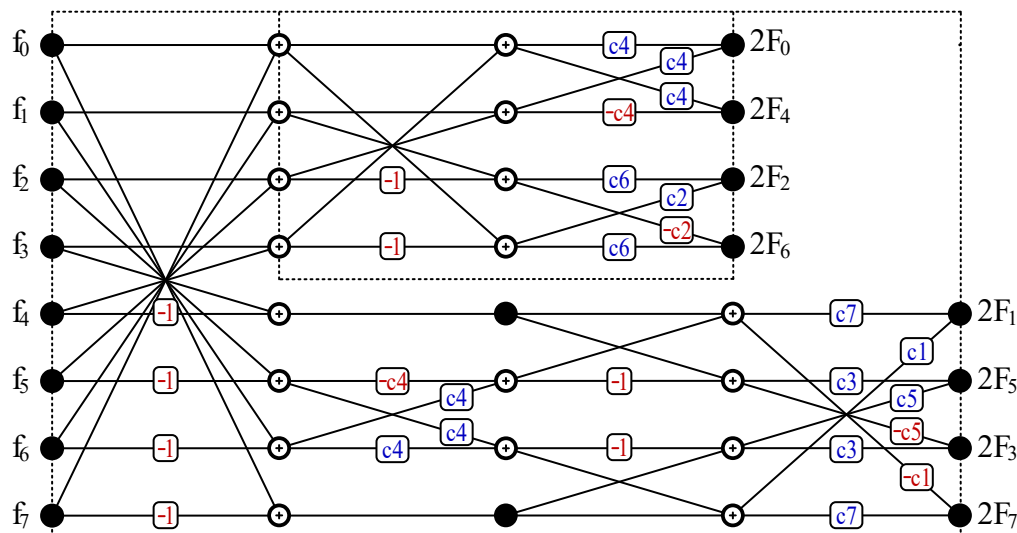


Figura 2.3. Diagrama algoritmului lui Chen pentru DCT rapidă în 8 puncte.

Contribuție lui Chen et al. constă în factorizarea formulei trigonometrice originale și rescrierea acesteia sub formă matriceală:

$$\begin{pmatrix} F_0 \\ F_2 \\ F_4 \\ F_6 \end{pmatrix} = \frac{1}{2} \cdot \begin{bmatrix} c4 & c4 & c4 & c4 \\ c2 & c6 & -c6 & -c2 \\ c4 & -c4 & -c4 & c4 \\ c6 & -c2 & c2 & -c6 \end{bmatrix} \cdot \begin{pmatrix} f_0 + f_7 \\ f_1 + f_6 \\ f_2 + f_5 \\ f_3 + f_4 \end{pmatrix}, \text{ unde } \begin{pmatrix} c1 \\ c2 \\ c3 \\ c4 \\ c5 \\ c6 \\ c7 \end{pmatrix} = \begin{pmatrix} \cos(\pi/16) \\ \cos(2\pi/16) \\ \cos(3\pi/16) \\ \cos(4\pi/16) \\ \cos(5\pi/16) \\ \cos(6\pi/16) \\ \cos(7\pi/16) \end{pmatrix} \quad (7)$$

$$\begin{pmatrix} F_1 \\ F_3 \\ F_5 \\ F_7 \end{pmatrix} = \frac{1}{2} \cdot \begin{bmatrix} c1 & c3 & c5 & c7 \\ c3 & -c7 & -c1 & -c5 \\ c5 & -c1 & c7 & c3 \\ c7 & -c5 & c3 & -c1 \end{bmatrix} \cdot \begin{pmatrix} f_0 - f_7 \\ f_1 - f_6 \\ f_2 - f_5 \\ f_3 - f_4 \end{pmatrix}$$

Diferiți algoritmi de calcul pentru DCT 1D au fost propuși de-a lungul anilor majoritatea utilizează 12 înmulțiri și 29 adunări, cu o operațiune de înmulțire mai mult decât limita teoretic posibilă. Excepție face algoritmul propus de Loeffler, care reușește reducerea numărului de înmulțiri la 11 fără a crește numărul de adunări [17].

Compresia video digitală a adoptat algoritmul propus de Arai, Agui și Naskajima [18] în 1988, care se bazează pe descoperirile lui Tseng [19] care demonstrează că DCT în 8 puncte poate fi calculată ca fiind partea reală a DFT în 16 puncte. Algoritmul necesită doar 5 înmulțiri și 29 adunări, dar dacă sunt necesare valorile explicite ale DCT în 8 puncte, acestea trebuie multiplicare utilizând constante scalare. Această etapă în compresia de imagine poate fi realizată în etapa de cuantizare fără a crește complexitatea calculului. Graficul fluxului de date se poate observa în Figura 2.4.

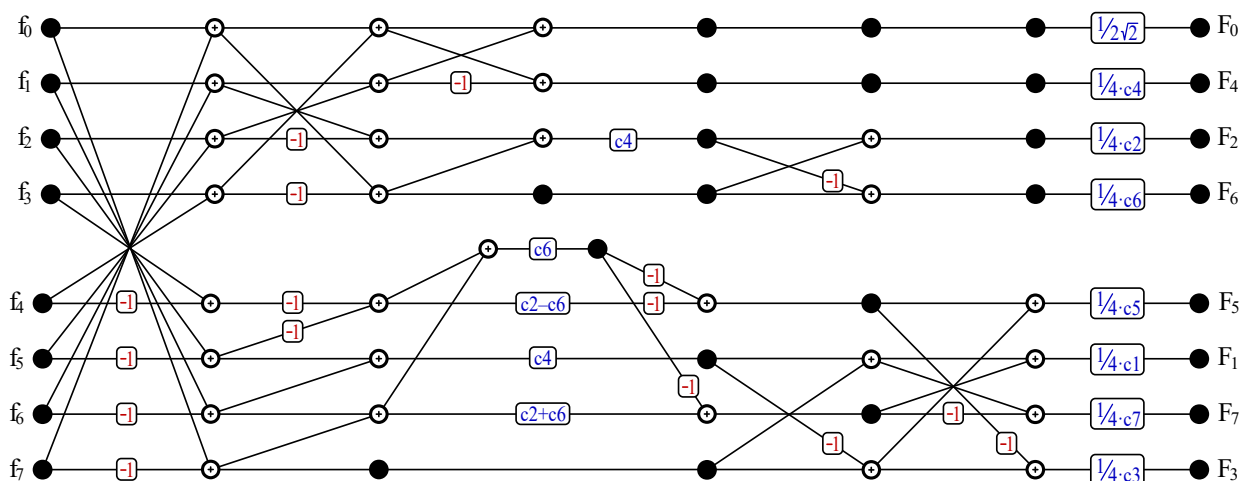


Figura 2.4. Diagrama *butterfly* a algoritmului lui Arai pentru DCT 1D în 8 puncte.

2.2.1.5. Transformatele subband și wavelet

DCT și alte transformate pe blocuri transformă partiționarea unei imagini în blocuri care nu se suprapun și procesează fiecare bloc separat. La rate de compresie foarte scăzute, coeficienții de transformare trebuie să fie cuantificați grosier și astfel va exista o eroare de reconstrucție semnificativă după decodificare. Această eroare este mai vizibilă la limitele (marginile) blocului cauzând o discontinuitate a imaginii și este cunoscută sub denumirea de artefact de blocare (engl. *blocking artefact*). O modalitate de a reduce acest artefact este să permită funcțiilor de bază să se destindă spre zero în aceste puncte sau să se suprapună peste blocurile adiacente. Ultima tehnică se numește transformarea ortogonală înfășurată [20]. Transformarea *wavelet* este un tip special de astfel de transformare și, prin urmare, este de așteptat să elimine artefactele de blocare.

La mijlocul anilor 1980, moment în care standardul JPEG original era în curs de dezvoltare, deși transformarea *wavelet* și predecesorul său, codarea *subband* erau cunoscute, nu a existat o metodă eficientă de codare a coeficienților de transformare *wavelet* care să fie comparabili cu DCT. De fapt, propunerile trimise comitetului JPEG au fost toate CODEC-uri bazate pe DCT, niciunul pe *wavelet*. De asemenea, aproape în același timp, din cele 15 propuneri către comitetul H.261, au existat 14 CODEC-uri bazate pe DCT și 1 metodă de cuantificare vectorială și nici unul pe *wavelet*. Astfel, la mijlocul anilor 1980, performanța de compresie cuplată cu un impuls considerabil deja în spatele DCT a determinat comitetul JPEG să adopte codificarea bazată pe DCT ca bază a standardului JPEG.

Codarea *subband* a fost introdusă inițial de Crochiere și colaboratorii în 1976 [21], și de atunci sa dovedit a fi o tehnică simplă și puternică de compresie a vorbirii și a imaginii. Principiul de bază este împărțirea spectrului de semnal în mai multe benzi de frecvență, apoi codarea și transmiterea fiecărei benzi separat. Acest lucru este foarte potrivit pentru codarea imaginilor. În primul rând, imaginile naturale tind să aibă un spectru de frecvență neuniform, cu cea mai mare parte a energiei concentrată în banda de frecvență inferioară. În al doilea rând, percepția omului asupra zgomotului tinde să scadă atât la frecvențe înalte, cât și la joase, ceea ce permite proiectantului să ajusteze distorsiunea de compresie în funcție de criteriile perceptive. În al treilea rând, deoarece imaginile sunt prelucrate în întregime și nu în blocurile artificiale, nu există o distorsionare a structurii blocurilor în imaginea codificată, așa cum se întâmplă în *encoder*-ele de imagine bazate pe transformări bloc, cum ar fi DCT.

Codarea sub-bandă, ca și transformarea Fourier, se bazează pe domeniul de frecvență în analiza imaginii, dar băncile sale de filtrare au o proprietate de de-corelare mai bună, care se potrivește mai bine imaginilor naturale. Acest lucru poate fi explicat după cum urmează: Funcțiile bazei Fourier sunt foarte precise în ceea ce privește frecvența, dar nu sunt precise din

punct de vedere spațial. Cu alte cuvinte, energia semnalului funcțiilor bazei Fourier nu este concentrată la o singură frecvență, ci răspândită în tot spațiul. Aceasta nu ar fi o problemă dacă pixelii de imagine au fost întotdeauna corelați. Cu toate acestea, în realitate, pixelii din imaginile de interes au, în general, o corelație scăzută, în special în ceea ce privește discontinuitățile imaginii, cum ar fi marginile. Spre deosebire de funcțiile bazei Fourier, bazele de sub-benzi nu numai că au o concentrație destul de bună a frecvenței, ci și sunt compacte spațial. Dacă marginile imaginii nu sunt prea împachetate, majoritatea elementelor de bază ale sub-benzii nu se vor intersecta cu acestea, astfel realizând în medie o decorare mai bună.

Transformarea *wavelet* este un caz special de codificare a sub-benzilor și a devenit foarte popular pentru codarea imaginilor statice și a celor în mișcare. În timp ce codarea sub-benzilor imaginilor se bazează pe analiza frecvenței, transformarea *wavelet* se bazează pe teoria aproximării. Cu toate acestea, pentru imaginile naturale care sunt netede local și pot fi modelate ca polinoame în bucăți, o funcție polinomială aleasă în mod corespunzător poate duce la o analiza a domeniului de frecvență, precum cea a subband. De fapt, *wavelet*-urile oferă un mijloc eficient de aproximare a acestor funcții cu un număr mic de elemente de bază.

2.2.1.6. Cuantizarea coeficienților

Cuantizarea este procesul de reducere a numărului de biți necesari pentru a stoca o valoare întreagă prin reducerea preciziei numărului. Având în vedere o matrice de coeficienți DCT, precizia coeficienților poate fi redusă în funcție de distanța față de coeficientul DC. Acest lucru se datorează faptului că, cu un coeficient DC este mai „departe”, cu atât elementul contribuie mai puțin la imaginea grafică și, prin urmare, cu mai puțin trebuie menținută precizia valorii sale [22]. Algoritmul împarte, în esență, fiecare coeficient DCT cu un număr întreg și elimină restul împărțirii. Rezultă o scădere a preciziei, dar sunt necesari mai puțini biți pentru a stoca valoarea acestuia.

Fiecare dintre cei 64 de coeficienți DCT este cuantificat în mod uniform pe baza unui tabel de cuantizare de 64 de elemente. În standardul JPEG de exemplu fiecare cuantă (element al tabelului de cuantizare) poate fi orice valoare întreagă de la 1 la 255, care specifică dimensiunea pasului cuantificatorului pentru coeficientul DCT corespunzător. Scopul cuantizării este de a obține o compresie suplimentară prin reprezentarea coeficienților DCT cu o precizie mai mare decât este necesară pentru a obține calitatea dorită a imaginii. Altfel spus, scopul acestei etape de procesare este de a elimina informații care nu sunt semnificative din punct de vedere al percepției vizuale a ochiului uman. Cuantizarea este o mapare multi-la-unu și, prin urmare, este fundamental cu pierderi. Etapa de cuantizare este principala sursă de pierderi în codificatoarele bazate pe DCT.

Cuantificarea este definită ca împărțirea fiecărui coeficient DCT la dimensiunea pasului său de cuantificare corespunzătoare, urmată de rotunjirea la cel mai apropiat număr întreg ($Qval$ este matricea cuantizată, val este matricea valorilor inițiale și Q este matricea de cuantizare):

$$Qval(i, j) = \frac{val(i, j)}{Q(i, j)} \quad (8)$$

Această valoare de ieșire este normalizată de dimensiunea pasului de cuantizare. De-cuantizarea este funcția inversă, în acest caz înseamnă pur și simplu că normalizarea este eliminată prin înmulțirea cu dimensiunea pasului, care returnează rezultatul la o reprezentare adecvată pentru intrarea în IDCT ($RQval$ este matricea de-cuantizată, $Qval$ este matricea valorilor inițiale și Q este aceeași matrice de cuantizare utilizată în operația de cuantizare):

$$DQval(i, j) = (Qval(i, j) \pm \frac{1}{2}) * Q(i, j) \quad (9)$$

Când scopul este de a comprima imaginea cât mai mult posibil fără artefacte vizibile, în mod ideal, fiecare dimensiune a pasului trebuie aleasă ca limită a pragului de percepție sau

„diferență doar vizibilă” pentru contribuția vizuală a funcției de bază cosinus corespunzătoare. Aceste praguri sunt, de asemenea, funcții ale caracteristicilor imaginii sursă, ale caracteristicilor de afișare și ale distanței de vizualizare. Pentru aplicațiile în care aceste variabile pot fi rezonabil de bine definite, pot fi efectuate experimente psiho-vizuale pentru a determina cele mai bune praguri. Experimentul descris de Lohscheller în [23] a condus la un set de tabele de cuantizare pentru imagini și dispozitive de afișare CCIR-601. Acestea au fost folosite experimental de membrii JPEG și apăr în standardul ISO ca cu scop informativ, și nu ca o cerință. Tabelul 2.1 conține tabelele de cuantificare exemplificative pentru componentele de luminanță și crominanțe prezentate în secțiunea K.1 a standardului JPEG. IJG (din engl. *Independent JPEG Group*) utilizează tabele similare cu o singură diferență: quanta pentru componenta DC de crominanță este modificată de la 17 la 16 pentru a oferi posibilitatea codării fără pierderi (lossless).

Tabelul 2.1. Tabelele de cuantizare pentru luminanță (stânga) și crominanțe (dreapta).

16	11	10	16	24	40	51	61	17	18	24	47	99	99	99	99
12	12	14	19	26	58	60	55	18	21	26	66	99	99	99	99
14	13	16	24	40	57	69	56	24	26	56	99	99	99	99	99
14	17	22	29	51	87	80	62	47	66	99	99	99	99	99	99
18	22	37	56	68	109	103	77	99	99	99	99	99	99	99	99
24	35	55	64	81	104	113	92	99	99	99	99	99	99	99	99
49	64	78	87	103	121	120	101	99	99	99	99	99	99	99	99
72	92	95	98	112	100	103	99	99	99	99	99	99	99	99	99

Un punct important de subliniat aici este faptul că tabelul de cuantizare utilizat pentru acest pas diferă între aproape toate pachetele software și camerele digitale. Producătorii de camere aleg în mod independent un nivel arbitrar de „calitate a imaginii” pentru a-l atribui matricei de cuantizare de 64 de valori pe care o elaborează, astfel încât numele nu pot fi comparate între mărci sau chiar modele ale aceluiași producător. Conform standardului JPEG tabelul de cuantizare utilizat este stocat ca parte a a fișierului rezultat.

Standardul JPEG permite utilizatorului specificarea unui factor de calitate exprimat în procente cu valoare întreagă între 1 (cea mai proastă calitate) și 100 (cea mai bună calitate). Acest factor de calitate este utilizat pentru a scala valorile din tabelele de cuantizare. IJG recomandă o curbă de scalare care pentru un factor de calitate de $Q=50$ tabelele de cuantizare sunt folosite așa cum sunt. Pentru un factor de calitate peste 50 (de la 50 la 100) scalarea procentuală a tabelor de cuantizare este dat de formula: $200-2*Q$ (se poate observa că pentru că la $Q=100$ scalarea este 0, ceea ce va face ca toate valorile din tabele să fie 1, oferind o pierdere minimă în urma procesului de cuantizare). Un factor de calitate sub 50 (între 1 și 49) este convertit în procentul de scalare $5000/Q$.

Partea a 3-a standardului JPEG, aprobat la sfârșitul anului 1995, introduce câteva extensii noi. Una dintre noile aditii este cuantizarea variabilă, care permite scalarea tabelului de cuantizare la diferite niveluri în diferite părți ale imaginii. În acest fel, părțile „mai critice” ale imaginii pot fi codificate la o calitate superioară față de părțile „mai puțin critice”. Un cod de semnalizare poate fi inserat la orice limită de bloc DCT pentru a seta un nou factor de scalare.

Adobe Photoshop de exemplu utilizează tabele de cuantizare proprii. Cele mai multe camere foto utilizează tabele de cuantizare adaptive, care se bazează pe imaginea procesată. Majoritatea producătorilor folosesc algoritmi brevetati pentru obținerea acestor tabelor de cuantizare adaptive. Acest lucru permite identificarea tipului dispozitivului utilizat pentru obținerea unei anumite imaginii.

2.2.2. Reducerea redundanțelor temporale

Utilizând diferențele dintre imaginile succesive, redundanța temporală este redusă.

Aceasta se numește codificare *inter-frame*. Pentru părțile statice ale secvenței de imagini, diferențele temporale vor fi aproape de zero și, prin urmare, nu sunt codificate. Acele părți care se schimbă între cadre, fie datorită variației de iluminare, fie mișcării obiectelor, conduc la o eroare semnificativă a imaginii, care trebuie codificată. Modificările imaginilor datorate mișcării pot fi reduse semnificativ dacă mișcarea obiectului poate fi estimată și diferența se face pe imaginea cu compensarea mișcării. Figura 2.5 prezintă eroarea *inter-frame* dintre cadrele succesive pentru o secvență de test „Claire” și corespondentul său compensat. Este clar că compensarea mișcării poate reduce substanțial eroarea *inter-frame*.



Figura 2.5. Inter-frame (stânga) și codarea cu compensarea mișcării (dreapta).

DPCM ***

2.2.2.1. Estimarea mișcării

Pentru a efectua o compensare a mișcării, trebuie mai întâi să se evalueze mișcarea obiectelor în mișcare. Aceasta se numește estimarea mișcării (ME – engl. *Motion Estimation*). Tehnica ME utilizată frecvent în toate CODEC-urile video standard este algoritmul de potrivire a blocurilor (BMA – engl. *Block Matching Algorithm*). Într-un BMA tipic, un cadru este împărțit în blocuri de pixeli $M \times N$ sau, mai uzual, blocuri pătrate de N^2 pixeli [24]. Apoi, pentru o deplasare maximă a mișcării de pixeli/cadru, blocul actual de pixeli se potrivește cu un bloc corespunzător la aceleași coordonate, dar în cadrul anterior, în interiorul ferestrei pătrate cu lățimea $N+2w$ (vezi Figura 2.6). Cea mai bună potrivire pe baza unui criteriu de potrivire duce la deplasare.

În criteriul de potrivire pot fi utilizate diferite măsuri cum ar fi funcția de corelație încrucișată (CCF – engl. *Cross-Correlation Function*), eroarea mediană-pătratică (MSE – engl. *Mean-Squared Error*) și eroarea absolută medie (MAE – engl. *Mean Absolute Error*) [25][26][27]. Pentru cea mai bună potrivire, în CCF corelația trebuie să fie maximizată, în timp ce în cele două din urmă distorsiunea trebuie minimizată. În practică, se utilizează atât MSE, cât și MAE, deoarece se crede că CCF nu ar oferi o urmărire bună a mișcării, mai ales atunci când deplasarea

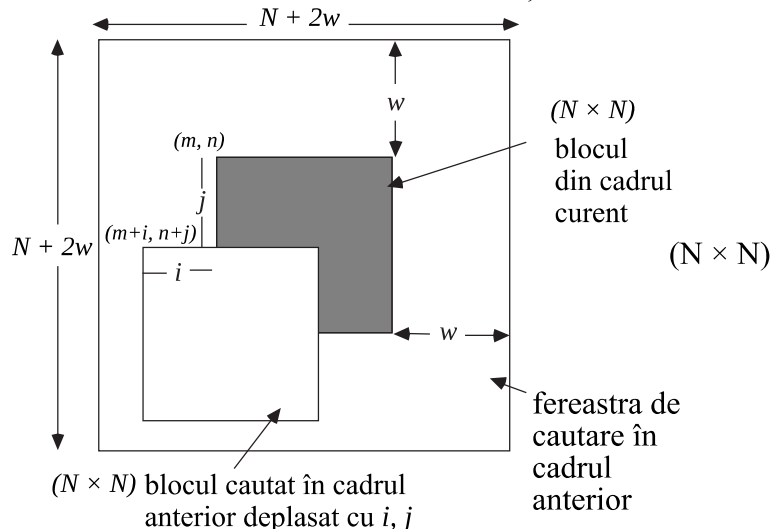


Figura 2.6. Fereastra de căutare pentru BMA.

nu este mare [27].

Pentru a găsi cea mai bună potrivire printr-o căutare completă, sune necesare $(2w+1)^2$ evaluări ale criteriului de potrivire. Pentru a reduce costul de procesare, MAE este de preferat față de MSE și, prin urmare, este utilizat în toate CODEC-urile video standard. Cu toate acestea, pentru fiecare bloc de pixeli N^2 , avem încă nevoie de a efectua $(2w+1)^2$ teste, fiecare cu aproape $2N^2$ adunări și scăderi. Acest lucru este încă departe de a fi adecvat pentru implementarea BMA în aplicațiile software. Măsurătorile complexității codificatorului video arată că ME cuprinde aproape 50-70% din complexitatea *encoder*-ului general [28]. Acest lucru, desigur, depinde de mișcare din scenă și de faptul dacă se utilizează un DCT rapid pentru derivarea coeficienților de transformare.

2.2.3. Codare pe lungime variabilă de biți

Pentru o reducere suplimentară a bandei necesare, coeficienții de transformare și coordonatele vectorilor de mișcare sunt codificați pe lungime variabilă de biți (VLC – engl. *Variable Length Coding*). În VLC, cuvintele de cod scurte sunt atribuite valorilor foarte probabile și cuvintelor de cod lung la cele mai puțin probabile. Lungimile codurilor ar trebui să varieze invers proporțional cu probabilitatea apariției diferitelor simboluri în VLC. Rata de biți necesară pentru a codifica aceste simboluri este inversa logaritmului probabilității, p , în baza 2 (biți), adică $\log_2 p$. Prin urmare, entropia simbolurilor, care reprezintă numărul mediu de biți necesari pentru a codifica simbolurile, poate fi calculată după formula:

$$H(x) = - \sum_{i=1}^n p_i \log_2 p_i \quad (10)$$

Există două tipuri de VLC, care sunt folosite în CODEC-urile video standard. Acestea sunt codificarea Huffman și codificarea aritmetică. Se remarcă faptul că codarea Huffman este un VLC simplu, dar compresia nu poate ajunge niciodată la fel de scăzută ca entropia datorită constrângerii că simbolurile atribuite trebuie să aibă un număr întreg de biți. Cu toate acestea, codificarea aritmetică poate aborda entropia, deoarece simbolurile nu sunt codificate individual [29]. Codificarea Huffman este folosită în toate CODEC-urile standard pentru a codifica coeficienții DCT cuantificați, precum și vectorii de mișcare. Codarea aritmetică este utilizată, de exemplu, în formatele JPEG, JPEG2000, H.263, H.264, H.265 și formarea și codarea imaginilor în format MPEG-4, unde se cere o comprimare suplimentară [30].

2.2.3.1. Codarea Huffman

Codificarea Huffman este cea mai cunoscută metodă VLC bazată pe statistici de probabilitate. Acesta atribuie un cod de ieșire fiecărui simbol, codurile de ieșire având o lungime începând cu 1 bit sau cu mult mai lungi decât simbolurile de intrare, în funcție de probabilitatea lor. Numărul optim de biți pentru fiecare simbol este $-\log_2 p$, unde p este probabilitatea unui simbol dat.

Cu toate acestea, deoarece cuvintele codificate trebuie să conțină un număr integrat de biți, codarea Huffman nu este optimă. De exemplu, dacă probabilitatea unui simbol este 0.33, numărul optim de biți care codifică acel simbol este de aproximativ 1.6 biți, dar schema de codare Huffman trebuie să atribuie codului 1 sau 2 biți. În ambele cazuri, în medie, va duce la mai multe biți în comparație cu entropia sa. Deoarece probabilitatea unui simbol devine foarte mare, codificarea Huffman devine foarte ne-optimală. De exemplu, pentru un simbol cu o probabilitate de 0.9, dimensiunea optimă a codului ar trebui să fie de 0.15 biți, dar codarea Huffman atribuie o valoare minimă de 1 bit de cod simbolului, care este de șase ori mai mare decât este necesar. Prin urmare, se poate vedea că resursele sunt irosite.

Pentru a genera codul Huffman pentru simboluri cu o probabilitate cunoscută de apariție, se efectuează următoarele etape:

- se poziționează toate simbolurile în ordinea probabilității de apariție a acestora;
- se combină succesiv fiecare două simboluri cu cea mai mică probabilitate de a forma un nou simbol compozit și se re-ordonează lista: acesta va genera un arbore, în care fiecare nod este probabilitatea tuturor nodurilor de sub el;
- se va urmări o cale către fiecare frunză, notând direcția de la fiecare nod.

De exemplu dacă se cunosc frecvențele de apariție ale simbolurilor generate de o sursă de informație S și faptul că sursa a generat în total m simboluri distincte ale unui alfabet, construcția arborelui binar devine foarte simplă. Practic primul pas constă în construirea unei „păduri” formată din m arbori. Fiecare arbore din pădurea inițială are un singur element care conține un simbol și frecvența lui de apariție. Toți cei m arbori din pădure sunt distincți (nu există doi arbori al cărui element să conțină același simbol). În continuare, la fiecare pas se elimină câte doi arbori din pădure pentru care costul rădăcinii este minim. Acești doi arbori vor fi sub-arborii unui nod al cărui cost va fi egal cu suma frecvențelor celor doi arbori eliminați din pădure. Arborele nou creat se adaugă la pădure. Algoritmul se termină în momentul în care în pădure mai rămâne un singur arbore, adică după efectuarea a $m-1$ iterații.

Din analiza acestui algoritm rezultă faptul că drumul de la rădăcină până la frunzele care au costul mai mic este mai mare decât drumul de la rădăcină până la frunzele care au costul mai mare. În Figura 2.7 se poate observa modul de construire al arborelui pentru șirul "To_be_or_not_to_be" generat de o sursă de informație S . Șirul are lungimea $n=18$ și are $m=8$ simboluri distincte. Simbolurile distincte sunt $\{b, e, n, o, r, t, T, _ \}$ și au frecvențele de apariție $2/18, 2/18, 1/18, 4/18, 1/18, 2/18, 1/18$ respectiv $5/18$.

Datorită faptului că toate fracțiile au același numitor, îl vom elimina, deci la sfârșitul algoritmului rădăcina arborelui obținut va avea costul n , unde n este numărul total de simboluri generate de sursa S . Eliminând numitorul comun se optimizează execuția unui program care implementează acest algoritm datorită faptului că operațiile cu numere întregi sunt executate mai rapid decât operațiile cu numere reale.

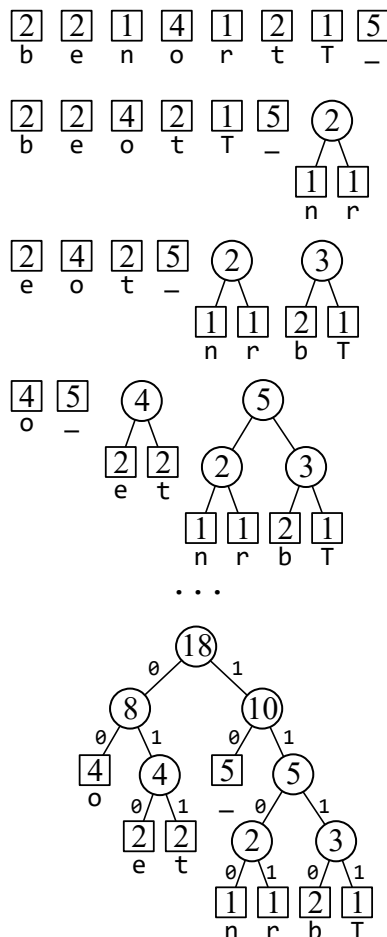


Figura 2.7. Exemplu de construcție a unui arbore Huffman.

Lungimea codului pentru un simbol generat de sursa S va fi egală cu lungimea drumului de la frunza corespunzătoare unui simbol până la rădăcina arborelui obținut folosind algoritmul lui Huffman. În final arborele obținut vom eticheta fiecare muchie dintre două noduri cu 0 dacă nodul fiu se află în stânga părintelui și cu 1 dacă nodul fiu se află în dreapta. Pentru a codifica simbolurile generate de sursa S , se va parcurge șirul generat și pentru fiecare simbol se va transmite la o ieșire E drumul de la rădăcină până la frunza corespunzătoare simbolului curent și astfel se transmit la ieșire etichetele muchiilor întâlnite pe drum.

Simbolurile din șirul dat anterior ca exemplu au codurile:

Tabelul 2.2. Codurile generate conform algoritmului Huffman.

Simbolul	Codul
b	1110
e	010
n	1100
o	00
r	1101
t	011
T	1111
—	10

Având codurile simbolurilor, șirul din exemplu se codifică astfel:

11110010111001010001101101100000111001100101110010

Lungimea codificării exemplului este de 50 biți (7 octeți). Dacă sursa de informație folosește codificarea ASCII, atunci lungimea în octeți a mesajului este 18, deci folosind această metodă se reduce spațiul utilizat cu 11 octeți. Pentru a putea decodifica acest mesaj, trebuie transmis și arborele creat, sau informațiile necesare pentru a putea fi construit. Dacă aceste informații lipsesc, nu se poate reconstitui mesajul inițial.

Arborele de codificare obținut cu algoritmul lui Huffman poartă numele de dicționar. Dicționarul este un termen general și în cazul altor tehnici de compresie. Dicționarul desemnează orice structură de date (sau informațiile necesare construirii ei) care se atașează șirului comprimat pentru ca programul de decompresie să poată reconstitui informația inițială. În cazul acestui algoritm o reprezentare eficientă a arborelui are dimensiunea de $2 \cdot m$ biți.

În cazul în care frecvențele de apariție ale simbolurilor sunt egale, iar numărul de simboluri distincte generate de o sursă S este egal cu numărul de simboluri ale alfabetului, atunci șirul simbolurilor nu se poate comprima folosind această metodă în felul cum a fost prezentat anterior, deoarece lungimea medie a codului unui simbol devine egală cu entropia maximă la care se mai adaugă și informația conținută de dicționar. În concluzie, cantitatea de informație transmisă va fi mai mare decât cea inițială. În acest caz, se folosesc variante mai complexe ale algoritmului.

2.2.3.2. Codarea Huffman canonică

În codarea Huffman standard, compresorul construiește un arbore Huffman pe baza numărului de apariții (sau a frecvențelor) simbolurilor care apar în secvența de date care urmează a fi comprimată și apoi atribuie fiecărui simbol cuvântul de cod indicat de parcurgerea căii de la rădăcină către nodul frunză asociat simbolului. De exemplu, dacă adoptăm convenția conform

căreia o „ramură” de la un nod la copilul său din stânga (respectiv, dreapta) este etichetată 0 (respectiv, 1), atunci dacă calea de la rădăcină la o anumită frunză este stânga, stânga, dreapta, stânga, dreapta, dreapta, rezultă pentru cuvântul de cod atribuit simbolului asociat valoarea 001011.

Codarea Huffman este un algoritm de comprimare a datelor fără pierderi în care fiecărui simbol din setul de date $\mathcal{S}$ se atribuie un cod de prefix de lungime variabilă. Caracterul cu cele mai puține apariții (frecvența de apariție cea mai mică) primește cel mai lung cod iar cel mai frecvent primește cel mai scurt cod. Codificarea datelor folosind această tehnică este foarte ușoară și eficientă. Cu toate acestea, decodificarea fluxului de biți generat folosind această tehnică este inefficientă. Decodificatoarele (sau decompresoarele) necesită cunoașterea mecanismului de codificare utilizat pentru a decoda datele codificate înapoi la simbolurile originale. Prin urmare, informațiile despre procesul de codificare trebuie să fie transmise decodorului împreună cu datele codificate ca un tabel de simboluri și codurile lor corespunzătoare. În codarea Huffman standard a unei cantități mari de date, acest tabel ocupă mult spațiu de memorie și, de asemenea, dacă un număr mare de caractere unice sunt prezente în setul de date, dimensiunea datelor comprimate (sau codificate) crește datorită prezenței tabelului de coduri.

Utilizarea arborelui Huffman standard are două ineficiențe critice în implementarea schemei de codare. În primul rând, fiecare nod al arborelui trebuie să stocheze fie referințe la nodurile sale secundare, fie simbolul pe care îl reprezintă. Acest lucru este costisitor în utilizarea memoriei și dacă există o proporție mare de simboluri unice în datele sursă, atunci dimensiunea arborelui de cod poate reprezenta o cantitate semnificativă din totalul datelor codificate. În al doilea rând, parcurgerea arborelui este costisitoare din punct de vedere computațional, deoarece necesită ca algoritmul să sară aleatoriu prin structura din memorie pe măsură ce fiecare bit din datele codificate este citit.

Prin urmare, pentru a face procesul de decodare eficient din punct de vedere computațional și pentru a menține totodată un raport (rată) de compresie bun(ă), au fost introduse codurile Huffman canonice. Codurile canonice Huffman abordează aceste două probleme prin generarea codurilor într-un format clar standardizat; tuturor codurilor pentru o lungime dată li se atribuie valorile succesiv. Aceasta înseamnă că, în loc să fie necesară stocarea structurii arborelui de cod pentru decompresie, sunt necesare doar lungimile codurilor, reducând astfel dimensiunea datelor codificate. În plus, deoarece codurile sunt secvențiale, algoritmul de decodare poate fi simplificat în mod dramatic, astfel încât să fie eficient din punct de vedere computațional.

În codarea Huffman canonică, se utilizează, astfel, doar lungimea secvenței de biți ale codurilor Huffman standard generate pentru fiecare simbol. Simbolurile sunt sortate mai întâi în funcție de lungimile lor de biți în ordine descrescătoare și apoi pentru fiecare lungime de biți, codurile sunt ordonate lexicografic¹. Primul simbol primește un cod care conține zerouri și are aceeași lungime cu secvența de biți inițială. Pentru simbolurile ulterioare, dacă simbolul are o lungime de biți egală cu cea a simbolului anterior, atunci codul simbolului anterior este incrementat cu 1 (unu) și atribuit simbolului curent. În caz contrar, dacă simbolul are o lungime de biți mai mare decât cea a simbolului anterior, după incrementarea codului simbolului anterior se adaugă zerouri până când lungimea devine egală cu lungimea secvenței de biți a simbolului curent și codul este apoi atribuit simbolului curent. Acest proces continuă pentru restul simbolurilor.

Pentru exemplul prezentat anterior cu secvența de date de intrare "To_be_or_not_to_be" s-a obținut valorile din Tabelul 2.2. Tabelul 2.3 de mai jos conține simbolurile Huffman standard obținute anterior ordonate crescător în funcție de lungimea simbolurilor și alfabetic în cadrul simbolurilor de aceeași lungime:

¹ În matematică, ordinea lexicografică este o generalizare a ordonării alfabetică a dicționarelor la secvențe de simboluri ordonate sau, mai general, a elementelor unei mulțimi total ordonate. Când se aplică numerelor, ordinea lexicografică reprezintă ordinea numerică crescătoare.

Tabelul 2.3. Codurile Huffman standard ordonate crescător funcție de lungime respectiv alfabetic.

Simbolul	Codul	Lungimea
_	10	2
o	00	2
e	010	3
t	011	3
b	1110	4
n	1100	4
r	1101	4
T	1111	4

Conform algoritmului pentru obținerea valorilor codurilor Huffman canonice primul simbol primește un codul cu valoarea 0 (zero) compus din zerouri și care are aceeași lungime cu secvența de biți inițială. Astfel valoarea codării pentru simbolul '_' va fi 00. Următorul simbol din tabel ('o') are aceeași lungime (2) cu primul simbol și valoarea codului va fi doar incrementată cu 1 față de valoare anterioară și altfel va simbolul 'o' va avea asociată secvența 01. Următorul simbol din tabel ('e') are lungime diferită (3) față de simbolul anterior și în plus față de incrementarea valorii cu 1 se va completa cu zerouri până la lungimea dorită și astfel codul simbolului 'e' devine 100. Algoritmul se repetă pentru restul simbolurilor din dicționar și la final se vor obține valorile din Tabelul 2.4:

Tabelul 2.4. Codurile Huffman canonice.

Simbolul	Lungimea	Codul canonic
_	2	00
o	2	01
e	3	100
t	3	101
T	4	1100
b	4	1101
n	4	1110
r	4	1111

Codurile Huffman canonice sunt astfel unic determinate doar pe baza lungimii codurilor Huffman standard. Pentru a decompresa o secvență de biți comprimată este necesară tabela de coduri utilizată la compresie, care pentru codurile Huffman canonice poate fi generată la decompresie pe baza unei tabele cu lungimile simbolurilor. Tabela cu lungimile simbolurilor poate fi salvată ca un vector de valori pentru lungimile tuturor simbolurilor în care simbolurile neutilizate vor avea coduri vide (lungime 0). Practic acest vector va conține lungimile simbolurilor ordonate lexicografic (valoarea indexului vectorului reprezintă valoarea numerică a simbolului din dicționar).

De exemplu pentru un o secvență de octeți de intrare pentru care valoarea maximă a lungimilor codurilor Huffman este 15 și pot fi astfel reprezentate ca valori fără semn pe 4 biți dimensiunea teoretică exprimată în biți a vectorului de lungimi a codurilor va fi $4 \cdot 256 = 1024$ biți. În realitate, datorită faptului că unitatea de bază pentru stocarea informației în sistemele de

calcul este octetul și nu bitul este necesară introducerea unui simbol suplimentar în lista simbolurilor care va avea frecvența de apariție 1 și care va reprezenta terminatorul secvenței de biți codați (comprimați). Astfel pentru un bloc generic de date compus din S simboluri pentru care numărul de biți necesar pentru reprezentarea lungimilor codărilor Huffman ale simbolurilor este n dimensiunea vectorului de lungimi ale simbolurilor va avea valoarea $(S+1)^*n$ exprimată în biți sau partea întreagă din $((S+1)^*n+7)/8$ ca valoare exprimată în octeți.

2.2.3.3. Arbori Huffman cu adâncime limitată

O problemă deloc simplă care necesită rezolvare în majoritatea implementărilor Huffman pentru seturi de date reale este limitarea adâncimii arborelui. Construcția Huffman standard nu limitează adâncimea arborelui generat. Dacă adâncimea arborelui ar fi limitată, acesta nu ar mai fi „optim”. Adâncimea maximă a unui arborelui Huffman este delimitată de *seria Fibonacci*, dar chiar și așa aceasta care este mai mare decât s-ar fi dorit.

Mai exact adâncimea maximă a unui arbore Huffman se obține în cazul în care frecvențele de apariție a simbolurilor sunt distribuite conform seriei Fibonacci. Cum suma distribuțiilor simbolurilor reprezintă dimensiunea blocului de date de comprimat rezultă că adâncimea maximă a unui arbore Huffman este dată de minimul dintre numărul simbolurilor și valoarea maximă a sumei primelor numere Fibonacci care se încadrează în dimensiunea blocului de date. Se poate demonstra cu ușurință că suma primelor numere Fibonacci până la al n -lea este egală cu al $(n+2)$ -lea număr Fibonacci minus 1 [31], conform formulei:

$$\sum_{i=0}^n F_i = F_{n+2} - 1 \quad (11)$$

Astfel adâncimea maximă a unui arbore Huffman pentru un bloc de date de dimensiune D cu s simboluri este minimul dintre s și n , unde n este valoarea maximă pentru care suma primelor n numere Fibonacci este strict mai mică față de D (sau $F_{n+2}-1 < D$, unde F_{n+2} este al $(n+2)$ -lea număr din seria lui Fibonacci).

Adâncimea arborelui Huffman trebuie limitată deoarece decodoarele Huffman rapide folosesc tabele de căutare (engl. *lookup tables*). Este posibilă utilizarea tabelelor cu mai multe niveluri pentru a reduce costul memoriei, dar un decodor foarte rapid, cum ar fi Huff0, utilizează un singur tabel, atât pentru simplitate, cât și pentru viteză. În acest caz, dimensiunea tabelului este un produs direct al adâncimii arborelui.

Pentru un spor de viteză vitezei și un management mai bun al memoriei, a trebuit selectată o limită. Inițial limita selectată a fost de 8 KiB pentru tabelul de decodare, care se potrivea bine la momentul respectiv în memoria cache de nivel 1 al procesoarelor Intel Pentium și lasă spațiu pentru a-l combina cu alte tabele dacă este necesar. De exemplu dacă un tabel de decodare folosește 2 octeți per celulă, acesta se traduce în celule de 4 KiB, deci o adâncime maximă a arborelui de 12 biți.

Cei 12 biți aleși inițial pentru comprimarea simbolurilor este în general prea puțin, cel puțin conform construcției unui arbore Huffman optim. Crearea unui arbore cu adâncime limitată este, prin urmare, o problemă practică ce trebuie rezolvată, iar adevărata provocare constă în atingerea acestui obiectiv cu un impact minim asupra raportului de compresie și în același timp să fie rapid?

Arbori Huffman cu adâncime limitată au fost studiați încă din anii 1960, așa că este disponibilă o literatură amplă pe această temă. Ceea ce este mai surprinzător este cât de complexe sunt soluțiile propuse și câte decenii au fost necesare pentru a se ajunge la o soluție optimă.

Aceste demersuri au început în 1961 cu Karp care propune o soluție cu timp exponențial [32], iar apoi Gilbert, în 1971 tot în timp exponențial [33]. Hu și Tan, în 1972 oferă o soluție de complexitate $O(n*D*2^D)$, unde n reprezintă numărul de biți pe care sunt reprezentate simbolurile, iar D este adâncimea maximă a arborelui [34]. În cele din urmă, o soluție în timp polinomial a

fost propusă de Garey în 1974, ce necesită $O(n^2 \cdot D)$ timp și folosește $O(n^2 \cdot D)$ spațiu [35]. În 1987, Larmore a propus ulterior o soluție îmbunătățită folosind $O(n^{3/2} \cdot D \cdot \log^{1/2} n)$ timp și spațiu [36]. O variantă revoluționară a apărut în 1990, când Larmore și Hirschberg propun algoritmul `Package_Merge`, un tip complet diferit de soluție folosind doar $O(n \cdot D)$ timp și $O(n)$ spațiu [37]. A devenit un algoritm de referință și a fost rafinat de câteva ori în următoarele decenii, cu contribuția notabilă a lui Mordecai Golin în 2008 [38].

Cele mai multe dintre aceste lucrări sunt pur și simplu greu de citit și, de obicei, și mai greu de aplicat pentru a dezvolta o soluție practică doar pe baza lecturii acestor lucrări. În practică, cele mai multe implementări rapide Huffman nu utilizează acești algoritmi. Sigur, atunci când este necesară o compresie optimă, algoritmul `Package_Merge` este preferat, dar în cele mai multe cazuri, a fi optim nu constituie ideea de bază, deoarece la urma urmei, algoritmul Huffman în sine este deja un compromis între optim și viteză. Urmând această logică, nu se dorește sacrificarea vitezei și a necesarului de memorie pentru a obține o soluție optimă, este necesar doar obținerea unei soluții *suficient* de bună, rapidă și ușoară.

De aceea, în multe codări Huffman se vor găsi soluții euristice *ieftine*. Un exemplu simplu: plecând de la un arbore Huffman clasic, se aplatizează toate frunzele la adâncimea maxim admisă, apoi se prelungesc suficiente frunze mai înalte până la valoarea maxim permisă pentru a compensa pentru modificarea inițială. Este o soluție rapidă, cu siguranță nu este optimă, dar în practică, diferența este mică și abia sesizabilă. Numai atunci când adâncimea copacului este foarte restrânsă se poate observa o diferență vizibilă.

Dacă se consideră simbolurile obținute după finalizarea unui arbore Huffman standard, sortate în ordine descrescătoare a numărului de biți necesari reprezentării fiecărui simbol, se vor aplatiza toate valorile mai mari decât `maxBits` (valoarea maxim admisă). Acest lucru va crea o „datorie”: un simbol care necesită `maxBits+1` biți creează o datorie de $1/2=0.5$ atunci când este forțat la `maxBits`. Un simbol care necesită `maxBits+2` creează o datorie de $3/4=0.75$ și așa mai departe. Ceea ce poate să nu fie total evident este că suma acestor datorii fracționale este în mod necesar un număr întreg. Aceasta este o consecință a pornirii de la un arbore Huffman rezolvat și poate fi dovedită prin recurență simplă: dacă lungimea naturală a arborelui Huffman este `maxBits+1`, atunci numărul de elemente la `maxBits+1` este în mod necesar par, în caz contrar, suma probabilităților poate să nu fie egală cu 1. Prin urmare, suma datoriei este în mod necesar un multiplu de $2 \cdot 0.5=1$, deci un număr întreg. Deducția poate fi generalizată pentru `maxBits+2` sau adâncimi mai mari.

La final va exista o datorie de *rambursat*. De fiecare dată când un simbol de la `maxBits-1` este „extins” la `maxBits`, se va plăti o datorie de valoare 1. Deoarece simbolurile sunt deja sortate cu o frecvență descrescătoare, este ușor să fie alese cele mai mici simboluri `maxBits-1` și să fie extinse la `maxBits`, până la rambursarea întregii datorii. Acest lucru este în esență ceea ce face ca euristica să fie *ieftină*.

Dar trebuie reținut faptul că retrogradarea unui simbol de la `maxBits-2` la `maxBits-1` rambursează nu o valoare de 1, ci o valoare de 2. Trecerea de la `maxBits-3` la `maxBits-2` rambursează o datorie de 4, și așa mai departe. Problema este, de fapt, dacă este mai eficientă retrogradarea un singur simbol `maxBits-2` față de două simboluri `maxBits-1`? Deoarece se operează cu un număr întreg de biți: se poate lua o decizie prin compararea suma aparițiilor celor două simboluri `maxBits-1` cu aparițiile simbolului `maxBits-2`. Oricare dintre cele două variante oferă cel mai mic cost în mai puțini biți este o soluție mai *bună*.

Această abordare poate fi scalată. Trebuie rambursată o valoare de 16 datorii? Un singur simbol la `maxBits-5` ar putea fi suficient sau 2 la `maxBits-4`. Prin recurență, fiecare simbol `maxBits-4` ar putea fi mai bine înlocuit cu doi `maxBits-3` și așa mai departe. Cea mai bună soluție poate fi identificată printr-un algoritm simplu de recurență.

Uneori, ar putea fi mai eficient să se depășească valoarea datoriei: de exemplu pentru o datorie de valoare 7, aceasta poate fi realizată fie pe baza formulei $4+2+1$ sau ca $8-1$. (-1 poate fi obținut prin promovarea celui mai bun simbol `maxBits` la `maxBits-1`). În teorie, ar trebui

comparate ambele variante și aleasă cea mai bună dintre acestea. Acest lucru ar duce la un algoritm optim. În practică, însă, rambursarea pozitivă a datoriei ($4+2+1$) este cel mai probabil cea mai bună, deoarece distribuția trebuie să fie sever răsturnată pentru ca soluția de depășire să se dovedească a fi mai bună.

Algoritmul devine puțin mai complex atunci când lipsesc unele rânduri (dimensiuni) de biți. De exemplu, trebuie rambursată o datorie de 2, dar nu mai există niciun simbol la $\text{maxBits}-2$. În acest caz, se mai pot folosi mai multe simboluri $\text{maxBits}-1$, dar poate că nu mai au rămas suficiente astfel de simboluri. În acest caz, singura soluție rămasă este să se depășească valoarea datoriei ($\text{maxBits}-3$) și să fie promovate suficiente elemente pentru a readuce datoria către zero.

În medie, implementarea acestui algoritm este destul de rapidă. Costul procesării sale este de neobservat, în comparație cu costul de decodare în sine, iar raportul de compresie final abia este afectat (în practică $<0,1\%$) în comparație cu adâncimea neconstrânsă a arborelui standard. Deci o astfel de abordare euristică oferă un compromis acceptabil și este aplicată în multe situații practice.

De exemplu, conform ITU T.81 care descrie formatul JPEG, vectorul BITS stochează „contorizările lungimii codului”. Crearea acestuia este descrisă în anexa K Figura K.2 din caietul de sarcini. Specificația JPEG se așteaptă la existența simbolurilor care necesită coduri Huffman cu lungimea de până la 32 de biți atunci când se realizează codarea. Cu toate acestea, limitează lungimea codului Huffman la maximum 16 biți atunci când datele sunt codificate. În acest scop, lungimile codului trebuie limitate la 16 biți. Procedura pentru aceasta este cuprinsă în anexa K Figura K.3.

2.2.3.4. Codificarea aritmetică

Codificarea Huffman poate fi optimă doar dacă probabilitatea simbolului este o putere întreagă a lui $1/2$, ceea ce nu se întâmplă decât în cazuri rarissime. Codificarea aritmetică este o tehnică de comprimare a datelor care codifică datele prin crearea unui șir de coduri, care reprezintă o valoare fracționară numerică între 0 și 1 [29]. Aceasta încurajează o separare clară între modelul de reprezentare a datelor și codificarea informațiilor cu privire la acest model. Un alt avantaj al codificării aritmetice este acela că ea elimină restricția pe care fiecare simbol trebuie să se traducă într-un număr întreg de biți, codificând astfel mai eficient. De fapt, se atinge entropia teoretică legată de eficiența de compresie pentru orice sursă. Cu alte cuvinte, codificarea aritmetică este o modalitate practică de a implementa codificarea entropiei.

Există două tipuri de modelare utilizate în codificarea aritmetică: modelul fix și modelul adaptiv. Modelarea este o modalitate de a calcula, în orice context, distribuția probabilităților pentru codul următor al simbolului. Trebuie să existe posibilitatea ca decodorul să producă exact aceeași distribuție de probabilități în același context. Spre deosebire de Huffman, codificarea aritmetică găzduiește ușor modele adaptive și este eficientă din punct de vedere computațional. Motivul pentru care comprimarea datelor necesită codificare adaptivă este acela că sursa de date de intrare se poate modifica în timpul codării, datorită mișcării și texturii.

În modelul fix, atât codificatorul, cât și decodorul cunosc probabilitatea atribuită fiecărui simbol. Aceste probabilități pot fi determinate prin măsurarea frecvențelor în eșantioanele reprezentative care urmează să fie codificate, iar frecvențele simbolurilor rămân fixe. Modelele fixe sunt eficiente atunci când caracteristicile sursei de date sunt apropiate de model și au fluctuații reduse.

În modelul adaptiv, probabilitățile atribuite se pot schimba pe măsură ce fiecare simbol este codificat, pe baza frecvențelor simbolurilor văzute până atunci. Fiecare simbol este tratat ca o unitate individuală și, prin urmare, nu este nevoie de un eșantion reprezentativ de text. Modelul actualizează distribuția inerentă, astfel încât predicția următorului simbol trebuie să fie apropiată de media reală de distribuție, făcând calea de la simbol la rădăcină mai scurtă.

Principiul de bază al codării aritmetice este acela de a folosi o scală în care sunt reprezentate intervalele de codare ale numerelor reale între 0 și 1. Aceasta este, de fapt, funcția

de densitate de probabilitate cumulată a tuturor simbolurilor care adaugă până la 1. Intervalul necesar pentru a reprezenta mesajul devine mai mic pe măsură ce mesajul devine mai lung și numărul de biți necesar pentru a specifica intervalul respectiv este mărit. În funcție de probabilitățile simbolului generate de model, mărimea intervalului este redusă prin simbolurile succesive ale mesajului. Simbolurile mai probabile reduc intervalul mai mic decât cele mai puțin probabile și, prin urmare, contribuie cu mai puțini biți la mesaj.

Pentru a explica modul în care funcționează codarea aritmetică, în exemplul de mai jos este folosit un model aritmetic fix pentru o ilustrare mai ușoară. Să presupunem că alfabetul este {a, e, i, o, u, !}, iar modelul fix este utilizat cu probabilitățile prezentate în tabelul 2.5. Odată ce probabilitatea simbolului este cunoscută, fiecare simbol individual trebuie să fie atribuit o porțiune a intervalului $[0, 1)$ care corespunde probabilității de apariție a acestuia în funcție de densitate cumulată. De asemenea, caracterul cu intervalul [*inferioară*, *superioară*) deține totul, de la o valoare *inferioară* până la, dar fără a include, valoarea *superioară*. Deci, alfabetul *u* cu probabilitatea 0.1, definit în intervalul cumulativ de $[0.8, 0.9)$, poate lua orice valoare de la 0.8 la 0.8999....

Tabelul 2.5. Exemplu pentru modelul fix pentru alfabetul {a, e, i, o, u, !}

Simbol	Probabilitatea	Intervalul
a	0.2	$[0.0, 0.2)$
e	0.3	$[0.2, 0.5)$
i	0.1	$[0.5, 0.7)$
o	0.2	$[0.7, 0.8)$
u	0.1	$[0.8, 0.9)$
!	0.1	$[0.9, 1.0)$

Cea mai importantă parte a unui mesaj codificat aritmetic este primul simbol care trebuie codificat. Folosind ca exemplu un mesaj "eaii!" care trebuie să fie codificat, primul simbol care urmează să fie codificat este 'e'. Prin urmare, mesajul codat final trebuie să fie un număr mai mare sau egal cu 0.2 și mai mic de 0.5. După ce primul cod este codat, știm că numărul inferior și numărul superior leagă acum intervalul pentru ieșire. Fiecare nou simbol care va fi codat va restricționa în continuare intervalul posibil al numărului de ieșire în timpul procesului de codificare.

Următorul caracter care urmează să fie codificat, 'a', se află în gama 0.0 – 0.2 în noul interval. Nu este primul număr care urmează să fie codificat, deci aparține domeniului corespunzător 0.0 – 0.2, dar în noua subordonare a lui $[0.2, 0.5)$. Aceasta înseamnă că numărul este acum limitat la intervalul $[0.2, 0.26)$, deoarece intervalul anterior a fost 0.3 ($0.5 - 0.2 = 0.3$) unități lungi și o cincime din acesta este 0.06. Următorul simbol care urmează să fie codificat, 'i', este în intervalul de $[0.5, 0.6)$, care corespunde la 0.5–0.6 în noua subordonare a lui 0.0–0.2 și dă intervalul mai mic $[0.23, 0.236)$. Aplicând această regulă pentru codarea caracterelor succesive, Tabelul 2.6 prezintă acumularea succesivă a intervalului mesajului codificat până acum.

Tabelul 2.6. Reprezentare a procesului codării aritmetice fixe.

Caracter	Interval
<i>inițial</i>	$[0.0, 1)$
e	$[0.2, 0.5)$
a	$[0.2, 0.26)$

i	$[0.23, 0.236)$
i	$[0.233, 0.2336)$
!	$[0.23354, 0.2336)$

Figura *** prezintă o altă reprezentare a procesului de codificare. Intervalul este extins pentru a umple întreaga gamă în fiecare etapă și marcat cu un interval care dă punctele finale ca număr. Intervalul final, $[0.23354, 0.2336)$, reprezintă mesajul "eaii!". Aceasta înseamnă că dacă transmitem un număr în intervalul de $0.23354 \geq x < 0.2336$, acel număr reprezintă întregul mesaj "eaii!".

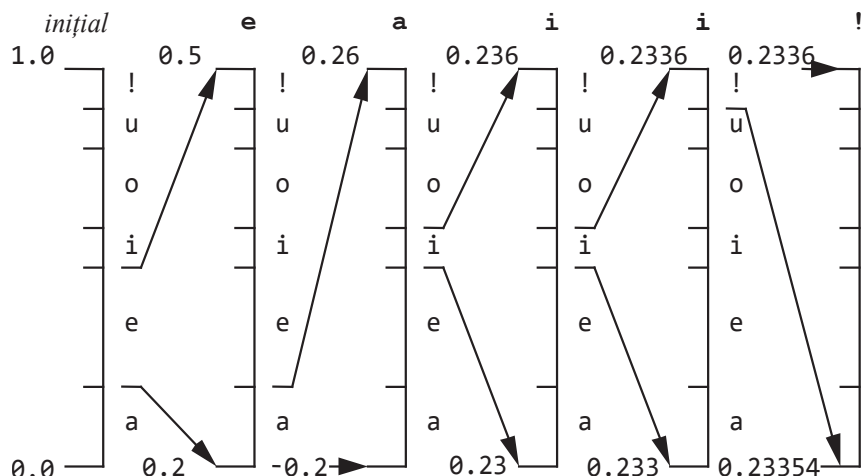


Figura 2.8. Reprezentarea procesului de codare aritmetică pentru mesajul eaii!

Având în vedere această schemă de codificare, este relativ ușor de observat cum în timpul decodificării elementele individuale ale mesajului "eaii!" sunt decodificate. Pentru a verifica acest lucru, fie un număr $x = 0.23355$ în intervalul de $0.23354 \geq x < 0.2336$. Decodorul, folosind aceleași intervale de probabilitate ca și codicatorul, efectuează o procedură similară. Începând cu intervalul inițial $[0, 1)$, numai intervalul $[0.2, 0.5)$ al lui 'e' cuprinde codul transmis de 0.23355 . Deci primul simbol nu poate fi decât 'e'. Similar cu procesul de codificare, intervalele de simboluri sunt apoi definite în noul interval $[0.2, 0.5)$. Aceasta este echivalentă cu definirea codului în intervalul inițial $[0, 1)$, dar compensarea codului cu valoarea inferioară și apoi scalarea în intervalul inițial. Adică, noul cod va fi $(0.23355 - 0.2) / (0.5 - 0.2) = 0.11185$, care este cuprins în intervalul $[0.0, 0.2)$ al simbolului 'a'. Astfel, al doilea simbol decodat este 'a'. Pentru a găsi cel de-al treilea simbol, trebuie găsit un nou cod în intervalul 'a', adică $(0.11185 - 0.0) / (0.2 - 0.0) = 0.55925$. Acest cod se încadrează în intervalul de $[0.5, 0.6)$ al simbolului 'i', iar noul cod rezultat după decodarea celui de-al treilea simbol va fi $(0.55925 - 0.5) / (0.6 - 0.5) = 0.5925$, care este din nou cuprins în $[0.5, 0.6)$. Prin urmare, al patrulea simbol va fi 'i'. Repetarea acestei proceduri va da un nou cod de $(0.5925 - 0.5) / (0.6 - 0.5) = 0.925$. Acest cod este cuprins în intervalul $[0.9, 1)$, care decodează simbolul '!'. Sfârșitul simbolului de decodare și procesul de decodare este astfel terminat.

Tabelul 2.7. Reprezentarea procesului de decodificare pentru codarea aritmetică.

Numărul encodat	Simbolul rezultat	Intervalul
0.23354	e	$[0.2, 0.5)$
0.233	a	$[0.0, 0.2)$
0.23	i	$[0.5, 0.6)$

0.23	i	[0.5, 0.6)
0.2	!	[0.9, 1.0)

2.2.3.5. Codificarea aritmetică adaptivă

În codificarea aritmetică adaptivă, probabilitatea asignată la simboluri se schimbă odată cu codarea fiecărui simbol [39]. Codarea binară (întreagă), se realizează prin atribuirea unei frecvențe de 1 pentru fiecare simbol la începutul codării (la inițializare). Pe măsură ce simbolul este codificat, frecvența acestui simbol este incrementată cu unu. Prin urmare, frecvențele simbolurilor sunt adaptate la numărul lor de apariții până la acel moment. Decodorul urmează o procedură similară. În fiecare etapă a codării, se efectuează un test pentru a vedea dacă frecvența cumulată (suma frecvențelor tuturor simbolurilor) depășește o valoare maximă. În acest caz, toate frecvențele sunt reduse la jumătate (minim 1) și codificarea continuă. Pentru o mai bună adaptare, frecvențele simbolurilor pot fi inițializate la o distribuție predefinită care să corespundă mai bine statisticii generale a simbolurilor. Pentru rezultate mai bune, frecvențele sunt actualizate de la numai N simboluri cele mai recent codate. S-a demonstrat că această metodă de adaptare cu o istorie anterioară limitată poate reduce banda de date necesară cu mai mult de 30% sub entropia de ordinul întâi al simbolurilor [40]. Motivul pentru aceasta este că, dacă evenimentele rare care au în mod normal o entropie mare pot apărea în clustere, atunci, în cadrul celor mai recente N evenimente, acestea devin acum evenimentele mai frecvente și, prin urmare, necesită un *bit-rate* mai mic.

Bibliografie

- [1] R. Albarino, „*Goldstein's Light Works at Southhampton*”, Variety, vol. 213, no. 12, 10 aug. 1966.
- [2] M. Blattner, R. Dannenberg, „*Multimedia Interface Design*”, ACM Press, 1992.
- [3] T. Vaughan, „*Multimedia: Making It Work (8th ed.)*”, McGraw Hill, 2011.
- [4] P.K. Andleigh, K. Thakrar, „*Multimedia Systems Design*”, Prentice Hall, 1996.
- [5] R. Steinmetz, K.Nahrstedt, „*Multimedia: Computing, Communications and Applications*”, Prentice Hall,, 1995.
- [6] S. Rosen, „*Signals and Systems for Speech and Hearing (2nd ed.)*”, BRILL, 2011.
- [7] R. Port, Audition for linguists [Online], Disponibil la adresa:
<https://www.cs.indiana.edu/~port/teach/sem08/hearing.for.linguists.final.html>, Accesat: 2018.
- [8] ***, Table Of Sound Pressure Levels [Online], Disponibil la adresa:
<http://www.sengpielaudio.com/TableOfSoundPressureLevels.htm>, Accesat: 2018.
- [9] N.J. Wade, H. Ono, „*From dichoptic to dichotic: historical contrasts between binocular vision and binaural hearing*”, Sage Journals, vol. 34 issue 6, pp. 645-668, 2005.
- [10] M. Ghanbari, „*Standard Codecs*”, The Journal of Engineering, 2011.
- [11] A.K. Jain, „*Fundamentals of Digital Image Processing*”, Prentice Hall, 1989.
- [12] W. Chen, C. Harrison Smith, S. C. Fralick, „*A fast computational algorithm for the discrete cosine transform*”, IEEE Transactions on Communications, vol. 25 , issue 9, pp. 1004–1009, sept. 1977.
- [13] M. Vetterli, H.J. Nussbaumer, „*Simple FFT and DCT algorithms with reduced number of operations*”, Elsevier, Signal Processing, vol. 6, issue 4, 1984.
- [14] R.M. Haralick, „*A storage efficient way to implement the discrete cosine transform*”, IEEE Transactions on Computers, vol. 25, issue 7, pp. 764–765, july 1976.
- [15] N. Ahmed, T. Natarajan, K.R. Rao, „*Discrete Cosine Transform*”, IEEE Transactions on Computers, vol. 23, issue 1, pp. 90–93, ian. 1974.
- [16] G.J. Sullivan, R.L. Baker, „*Efficient quadtree coding of images and video*”, IEEE Transactions on Image Processing, vol. 3, issue 3, pp. 327–331, may 1994.
- [17] C. Loeffler, A. Ligtenberg, G.S. Moschytz, „*Practical fast 1-D DCT algorithms with 11 multiplications*”, International Conference on Acoustics, Speech, and Signal Processing, vol. 2, pp. 988–991, may 1989.
- [18] Y. Arai, T. Agui, M. Nakajima, „*A Fast DCT-SQ Scheme for Images*”, IEICE TRANSACTIONS, no. 11, pp. 1095-1097, nov. 1988.
- [19] B.D. Tseng, W.C. Miller, „*On Computing the Discrete Cosine Transform*”, IEEE Transactions on Computers, vol 27, issue 10, pp. 966–968, oct. 1978.
- [20] H.S. Malvar, D.H. Staelin, „*The LOT: Transform Coding Without Blocking Effects*”, IEEE Transactions on Acoustics, Speech, and Signal Processing, vol 37, no. 4, pp. 553–559, 1989.
- [21] B.E. Usevitch, „*A tutorial on modern lossy wavelet image compression: foundations of JPEG 2000*”, IEEE Signal Processing Magazine, vol. 18 , no. 5, pp. 22–35, 2001.
- [22] M. Nelson, „*Data Compression Book*”, Wiley, aug. 1991.
- [23] H. Lohscheller, „*A Subjectively Adapted Image Communication System*”, IEEE Transactions on Communications, vol. 32, issue 12, pp. , dec. 1984.
- [24] T. Ishiguro, K. Iinuma, „*Television bandwidth compression transmission by motion-compensated interframe coding*”, IEEE Communications Magazine, vol. 20 , issue 6, pp. 24 - 30, 1982.
- [25] S. Kappagantula; K. R. Rao, „*Motion Compensated Predictive Coding*”, The International Society for Optics and Photonics, vol. 0432, pp. , 1984.

-
- [26] H.C. Bergmann, „*Displacement estimation based on the correlation of image segments*”, IRE Conference on the Electronic Image Processing, 1982.
 - [27] J.R. Jain, A.K. Jain, „*Displacement Measurement and Its Application in Interframe Image Coding*”, IEEE Transactions on Communications, vol. COM-29, no. 12, pp. 1799–1808, 1981.
 - [28] T. Shanableh, M. Ghanbari, „*Heterogeneous video transcoding to lower spatio-temporal resolutions and different encoding formats*”, IEEE Transactions on Multimedia, vol. 2, issue 2, pp. 101–110, 2000.
 - [29] G.G. Langdon, „*An introduction to arithmetic coding*”, IBM Journal of Research and Development, vol. 28, issue 2, pp. 135–149, 1984.
 - [30] D. Marpe, H. Schwarz, T. Wiegand, „*Context-based adaptive binary arithmetic coding in the H.264/AVC video compression standard*”, IEEE Transactions on Circuits and Systems for Video Technology, vol. 13, no. 7, pp. 620–636, 2003.
 - [31] E. Lucas, „*Number theory*”, Paris Gauthier-Villars, 1891.
 - [32] R. Karp, „*Minimum-redundancy coding for the discrete noiseless channel*”, IRE Transactions on Information Theory, vol. 1, issue 1, pp. , ian. 1961.
 - [33] E. Gilbert, „*Codes based on inaccurate source probabilities*”, IEEE Transactions on Information Theory, vol. 17, issue 2, pp. , mai 1971.
 - [34] T.C. Hu, K.C. Tan, „*Path Length of Binary Search Trees*”, SIAM Journal on Applied Mathematics, vol. 22, issue 2, pp. , mar. 1972.
 - [35] M.R. Garey, „*Optimal Binary Search Trees with Restricted Maximal Depth*”, SIAM Journal on Computing, vol. 3, issue 2, pp. 101–110, iun. 1974.
 - [36] L.L. Larmore, „*Height restricted optimal binary trees*”, SIAM Journal on Computing, vol. 16, issue 6, pp. 1115–1123, dec. 1987.
 - [37] L.L. Larmore, D.S. Hirschberg, „*A fast algorithm for optimal length-limited Huffman codes*”, Journal of the ACM, vol. 37, issue 3, pp. 464–473, iul. 1990.
 - [38] M. Golin, Y. Zhang, „*A Dynamic Programming Approach To Length-Limited Huffman Coding*”, Computing Research Repository, , pp. , iun. 2008.
 - [39] I.H. Witten, R.M. Neal, J.G. Cleary, „*Arithmetic coding for data compression*”, Communications of the ACM, vol. 30, no 6, pp. 520–540, 1987.
 - [40] M. Ghanbari, „*Arithmetic coding with limited past history*”, Electronics Letters, vol. 27, no. 13, pp. 1157 - 1159, 1991.